

VCF-RDFizer: A Conversion Tool to Represent Variant Data as a Knowledge Graph

Elias Crum^{ORCID^{1,2*}}, Bart Buelens^{ORCID¹}, Ruben Taelman^{ORCID²}, Gökhan Ertaylan^{ORCID¹}

²IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium.

¹Digital Biological Systems, Flemish institute for Technological Research (VITO), Boeretang 200, 2400, Belgium.

*Corresponding author(s). E-mail(s): elias.crum@ugent.be;

Abstract

Purpose: The Variant Call Format (VCF) is the dominant standard for representing genomic variation and underpins population genomics, clinical sequencing, and precision-medicine workflows. As VCF datasets are increasingly interpreted together with sample metadata, clinical observations, public biomedical resources, and governance information, a Resource Description Framework (RDF) representation can provide a common graph model for linkage and declarative querying.

Methods: We present VCF-RDFizer, a command-line tool distributed through PyPI, Conda-Forge, and Docker that converts VCF into RDF using a public VCF-oriented vocabulary and modular RML mappings. In addition to raw N-Triples and general-purpose gzip/Brotli artifacts, the current workflow produces two queryable compressed representations: HDT and COTTAS. A storage-aware chunk pipeline incrementally reads a plain or concatenated-gzip RDF aggregate, materializes only one newline-safe uncompressed chunk at a time, converts that chunk into the selected representation(s), deletes the chunk, and merges the representation parts. We evaluate VCF-RDFizer v3.1.0 on a ten-file public corpus, record and sample scaling series, and purpose-built fixtures. Validation combines native representation decoding with thirteen paired SPARQL/VCF-parser queries and SHACL checks on designated inputs; a mutation experiment tests detection of 113 injected corruptions.

Results: Eight corpus files converted as 250,000-record slices, and a ninth, a 2.36 GB Genome in a Bottle VCF, converted whole (657M triples). The tenth, a 2,504-sample cohort, was exercised through 10,000-record slices. COTTAS was the smallest queryable artifact throughout, at $0.37\text{--}0.55\times$ the gzip-compressed

N-Triples, while uncompressed HDT was $1.36\text{--}1.82\times$ larger than those N-Triples. Space-optimized storage cut peak workspace by $7.7\text{--}9.2\times$ for a 3–5% time cost, and memory stayed at 1.0–1.7 GB across a hundredfold change in input. Of 988 paired comparisons, 984 were exactly equal and 4 were verified not-applicable; all completed cross-engine comparisons agreed across both sample encodings and all three artifact types. The queries alone detected 96 of 113 injected corruptions; adding the shapes detected all 113. Per question, SPARQL answered $2.5\text{--}2300\times$ faster than a VCF parser; a single parser pass remained faster for all thirteen questions together.

Conclusion: VCF-RDFizer provides a reproducible route from VCF to interoperable and optionally queryable compressed RDF, supporting both semantic integration and testable preservation of core variant and genotype information.

Keywords: Knowledge Representation, Genomic Variant Data, Variant Call Format, RDF, SPARQL, HDT, COTTAS, Semantic Validation

Revision-marking key. Blue text denotes additions or substantively revised passages. Red strikethrough, where used, denotes superseded wording. Orange [AUTHOR QUERY] statements identify information that is not supported by the supplied artifacts and must be resolved before submission.

1 Introduction

Variant Call Format. The Variant Call Format (VCF) is the dominant standard for representing and exchanging genomic variants across research and clinical sequencing pipelines, including pharmacogenomics and precision medicine applications [1–3]. As clinical sequencing expands, variant datasets are increasingly interpreted alongside external biomedical resources, patient or sample metadata, and governance constraints controlling access to sensitive genomic information. Operational semantic-genomics initiatives already demonstrate the value of linking variant observations to curated biomedical databases and controlled-access frameworks [4, 5].

These emerging use cases expose limitations of the traditional VCF representation. First, clinical interpretation frequently requires linking variants to external knowledge sources such as protein annotations, disease databases, or pharmacogenomic resources. While VCF supports annotation fields, it provides limited mechanisms for structured, machine-interpretable linkage between variant records and external resources. Second, genomic datasets frequently require fine-grained privacy controls, such as restricting access to particular genomic regions or subsets of variants, which the VCF specification does not natively support. Third, querying variant collections across multiple VCF files, for example, identifying clinically relevant mutations or retrieving variants within genomic position intervals, typically relies on specialized command-line utilities (e.g., bcftools [6]) or custom pipelines rather than interoperable, declarative query mechanisms.

In other domains, semantic web and policy frameworks, such as Linked Data [7] and the Open Digital Rights Language (ODRL) [8], enable structured linkage between

datasets, external knowledge resources, and access control policies. Applying similar approaches to genomic variant data could enable richer integration with biomedical knowledge bases and more expressive query capabilities.

Resource Description Framework and Knowledge Graphs. The Resource Description Framework (RDF) [9] provides a widely adopted graph-based data model for representing relationships between entities identified by globally unique identifiers (IRIs). Such graph-structured data can be organized as *knowledge graphs*, in which nodes represent entities of interest and edges capture relationships between them, with the graph serving to accumulate and convey knowledge [10]. For variant data, RDF enables (i) structured linkage between variant data and external biomedical resources [11–13], (ii) native declarative querying via the SPARQL query language [14], and (iii) fine-grained privacy policy enforcement over variant data and linkages [8]. In practice, this could enable, for instance, a user to query a collection of VCF-derived datasets to identify variants whose alternative alleles occur in a specific gene, are associated with a given phenotype, and are accessible under particular policy restrictions.

From VCF to RDF. Transforming VCF into RDF requires *semantic lifting*: converting the row-based structure of VCF files into graph statements. Relevant VCF elements such as genomic position, alleles, genotypes, and sample identifiers are assigned IRIs, and fields are translated into subject–predicate–object triples. Ontology-aligned predicates provide consistent semantics which enable datasets to be queried and linked.

This transformation requires two components: (i) a semantic vocabulary describing VCF concepts and relationships, and (ii) a mapping process that converts VCF file content into RDF using this vocabulary. Such mappings must be systematic and reproducible due to the structural heterogeneity permitted by the VCF specification [1]. INFO fields may contain semi-structured annotations, FORMAT and genotype columns vary across datasets, and multi-sample files aggregate multiple genotypes within shared variant records, complicating the creation of consistent semantic representations.

Several efforts have explored semantic representations of VCF and genomic-variation data, ranging from direct VCF-to-RDF converters to systems that expose genomic variation through RDF and SPARQL interfaces.

Early approaches include the *rdf-vcf* reader and the web-based *VCF2RDF*, the latter introducing an isomorphic mapping between VCF and RDF/JSON-LD [17, 21]. The 2013 BioHackathon produced another early *vcf2rdf*, a JRuby converter that describes positions with FALDO and alleles and genotypes with tool-specific terms [18]. Dedicated conversion utilities include JVarkit’s *VcfToRdf*, the Rust-based *VCF2RDF* utility developed for TOGOvar [4, 15], and the *vcf2rdf* component of SPARQLing Genomics [16, 25]. More general genomic-format tools also reach RDF: *BioInterchange* emits the Genomic Feature and Variation Ontology (GFVO) from GFF3, GVF and VCF, although its VCF reader extends its GFF3 reader, so records become generic features, and it reads VCF 4.1–4.2 only [19]; and *bio-vcf* can serialize VCF-derived information as RDF or JSON-LD through configurable output templates [20]. VCF-derived graphs are also built inside larger knowledge-graph pipelines. VariantKG annotates VCFs with SnpEff, converts them with SPARQLing Genomics’

Table 1 VCF-to-RDF converters and related semantic genomic-variation workflows, compared on the capabilities this study targets. Assessments are based on each system’s publicly available documentation.

Tool / workflow	CLI	RDF model	Reprod. workflow	Metadata + genotype	Extensible mappings	RDF compr.	Public release
<i>Dedicated VCF-to-RDF converters</i>							
VCF-RDFizer (this study)	✓	✓	✓	✓	✓	✓	✓
TogoVar VCF2RDF [4]	✓	(✓)	✓	(✓)	(✓)	–	✓
JVarkit VcfToRdf [15]	✓	(✓)	✓	(✓)	–	–	✓
SPARQLing Genomics [16]	✓	(✓)	✓	✓	(✓)	–	(✓)
VCF2RDF (web) [17]	(✓)	✓	(✓)	(✓)	–	–	(✓)
BioHackathon vcf2rdf [18]	✓	(✓)	(✓)	(✓)	–	–	(✓)
<i>General genomic-format tools</i>							
BioInterchange [19]	✓	✓	✓	(✓)	–	–	✓
bio-vcf [20]	✓	–	✓	(✓)	✓	–	✓
ruby-rdf/rdf-vcf [21]	✓	–	(✓)	(✓)	–	–	✓
<i>Knowledge-graph construction pipelines</i>							
VariantKG ^a [22]	(✓)	(✓)	(✓)	✓	–	–	(✓)
AgroLD VCF2RDF ^b [23]	–	(✓)	–	–	–	–	(✓)
<i>Virtual semantic access</i>							
Semantic Beacons [24]	–	(✓)	(✓)	(✓)	✓	–	(✓)

✓ supported and documented; (✓) partial or limited; – not found in public documentation. **CLI**: command-line use in pipelines. **RDF model**: documented vocabulary for the output. **Reprod. workflow**: documented, reproducible VCF-to-RDF route. **Metadata + genotype**: header metadata kept with genotype and sample data. **Extensible mappings**: rules editable without changing the converter. **RDF compr.**: integrated compressed RDF output. **Public release**: distributed package, executable or container. ^aConverts VCF through SPARQLing Genomics’ vcf2rdf after SnpEff annotation, then adds its own ontology for CADD scores and patient metadata. ^bNamed in [23] as part of the AgroLD ETL repository, which contained no VCF conversion code when checked (September 2026); ratings reflect what could be verified.

vcf2rdf, and adds its own ontology for CADD scores and patient metadata to support graph machine learning [22]; AgroLD names a VCF2RDF component of the ETL that feeds its plant-science knowledge graph [23], but that component’s code could not be located in the cited repository. In contrast to these materializing approaches, *Semantic Beacons* uses declarative RML mappings to expose genomic-variation data obtained through the Beacon API as RDF, enabling federated SPARQL access without functioning as a conventional VCF-to-RDF converter [24].

Table 1 compares these approaches according to their semantic modeling, transformation workflow, support for metadata and genotype information, extensibility, compression, and software distribution. The comparison illustrates that previous systems address different portions of the VCF-to-semantic-data workflow. Some prioritize lightweight and efficient conversion, others provide highly customizable serialization or virtual semantic access, while several expose only a subset of VCF structure or rely on implementation-specific RDF models. VCF-RDFizer is designed to combine these concerns within a single reproducible workflow by being grounded in a publicly documented VCF semantic model, declarative and extensible mappings, systematic representation of record-, metadata-, and sample-level information, and integrated production of both conventional and compressed queryable RDF representations.

Open Challenges.

First, existing VCF-to-RDF approaches differ substantially in how explicitly and completely they define the semantics of the generated RDF. Some systems document an RDF representation or mapping of VCF concepts, such as the isomorphic VCF-to-RDF model of the web-based VCF2RDF [17] or BioInterchange’s GFVO output [19], while others use lightweight or implementation-specific RDF structures optimized for particular conversion or application requirements [4, 15, 18]. Pipelines such as VariantKG inherit whichever structure their converter emits and extend it for their own application [22]. More general tools such as *bio-vcf* allow users to define custom RDF serializations, but do not prescribe a reusable semantic model of VCF structure.

Consequently, a remaining challenge is the availability of a standalone, publicly documented vocabulary that systematically captures the structural semantics of VCF itself, including file- and header-level metadata, variant records, INFO and FORMAT definitions and values, sample-level calls, and genotype relationships. Existing genomic ontologies and data models such as GA4GH Variant Representation [26], FALDO [27], the Sequence Ontology [28], GFVO [19], GENO, and SNPO provide important semantics for genomic locations, variation, and biological entities, but were not designed to represent the complete structure and serialization semantics of a VCF document. A dedicated VCF semantic layer can therefore complement these resources while reusing their established biological concepts where appropriate.

Second, extensibility is supported through different mechanisms across existing converters, but semantic transformation rules are often coupled to implementation logic, configuration files, or application-specific templates. For example, TOGOvar’s *VCF2RDF* exposes configurable namespaces, INFO-field selection, and reference mappings through YAML configuration [4], while *bio-vcf* provides highly flexible output templates capable of generating customized RDF representations. Such mechanisms are useful for adapting output, but the resulting transformation semantics remain closely associated with the particular software or template implementation.

Declarative mapping languages provide a complementary approach by representing transformation rules independently from the execution engine. RML (RDF Mapping Language) [29] supports mappings from heterogeneous data sources into RDF and enables mapping definitions to be reused, inspected, versioned, and extended independently of the software executing them. This approach is also employed by Semantic Beacons, which uses RML mappings to transform Beacon API responses into RDF for on-demand semantic access [24]. The RML ecosystem additionally includes engines such as RMLStreamer [30], which support scalable execution of declarative mappings. For VCF conversion, separating the semantic model and mapping rules from procedural parsing logic provides a mechanism for adapting dataset-specific INFO, FORMAT, metadata, or ontology alignments without requiring modifications to the core conversion implementation.

Third, most existing VCF-to-RDF workflows do not integrate strategies for addressing the storage expansion introduced by semantic lifting. Expanding compact genomic formats such as VCF into explicit RDF triples can substantially increase storage requirements, particularly because identifiers and structural relationships that are implicit in VCF become explicit and may be repeated across large numbers of triples [9].

Compression strategies therefore play an important role, but archival and queryable compression address different requirements. General-purpose compressors such as gzip and Brotli [31] can substantially reduce storage and transfer costs, but the resulting artifacts ordinarily require decompression before they can be queried as RDF. In contrast, RDF-specific representations such as HDT (Header–Dictionary–Triples) [32] combine compression with indexed triple-pattern access. More recently, COTTAS represents RDF as a columnar triple table using Apache Parquet and block-range indexes, enabling triple-pattern evaluation directly over the compressed representation [33]. Integrating both archival and queryable representations within a VCF-to-RDF workflow therefore allows users to select storage strategies according to whether the resulting genomic knowledge graph is intended primarily for distribution, long-term storage, direct querying, or a combination of these purposes.

Study Aims. Taken together, these limitations motivate the development of a reusable and reproducible VCF-to-RDF transformation tool whose output can be independently validated. Here we present *VCF-RDFizer*, which integrates a publicly documented vocabulary, modular RML-based mappings, storage-aware conversion, general-purpose compression, and queryable HDT and COTTAS representations. As summarized in Table 1, VCF-RDFizer combines capabilities that are typically disconnected in existing approaches: a standalone CLI <https://github.com/ecrum19/VCF-RDFizer>, the public VCF Core Vocabulary <https://w3id.org/vcf-core/vocab#> [34], modular declarative mappings, a reproducible pipeline, queryable compressed outputs, and public packaging <https://pypi.org/project/vcf-rdfizer/>. In addition to measuring conversion and compression costs, this study defines a paired SPARQL/VCF-parser workload to determine whether the RDF representation reproduces established variant and genotype summaries exactly.

2 VCF-RDFizer CLI Tool

VCF-RDFizer is a command-line tool that converts input VCF files into semantically enriched RDF using a reproducible mapping pipeline. The workflow integrates pre-processing, mapping, RDF conversion, compression/decompression functionalities, graph validation functionalities, graph data linking plug-in architecture, and other features. An overview of these workflows is shown in Figure 1.

2.1 Semantic Model

VCF-RDFizer emits RDF in the VCF Core Vocabulary (`vcfc:`, <https://w3id.org/vcf-core/vocab#>) [34], a separately maintained vocabulary that models the VCF specification itself, version by version, for VCFv4.1 through VCFv4.5. It is described and evaluated in its own right in a companion paper [35]; here it is the target model of the conversion. Its terms align with established resources for genomic coordinates (FALDO [27]), variant typing (the Sequence Ontology [28]), genotype and sample constructs (HERO [36]), and features of the GA4GH VRS schema v2.0.1 [37], so that VCF-specific structure is kept without duplicating mature ontology terms.

The model preserves three information layers: file and header assertions, variant-record assertions, and sample and genotype assertions. Sample-level data can be

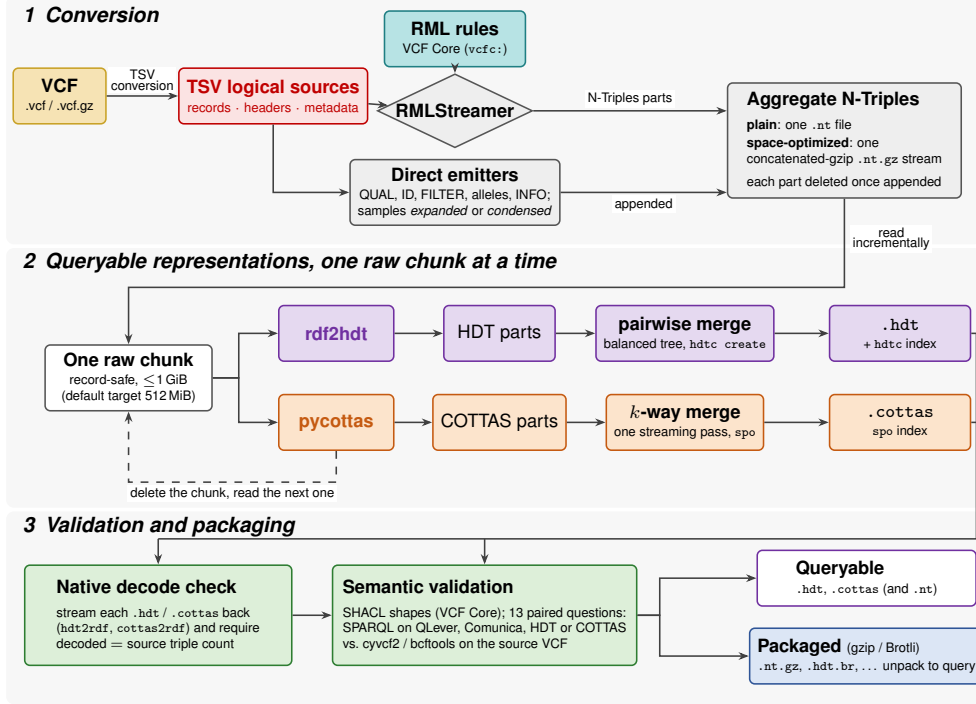


Fig. 1 VCF-RDFizer v3.1.0 pipeline. (1) **Conversion:** the VCF is split into three TSV logical sources; RMLStreamer applies the RML rules to emit VCF Core N-Triples, and direct emitters add the row-dependent fields RML cannot type per row (QUAL, ID, FILTER, alleles, structured INFO) and the samples in the *expanded* or *condensed* profile. Every part is appended to one aggregate and then deleted: a plain .nt file, or in space-optimized mode a concatenated-gzip .nt.gz stream. (2) **Queryable representations:** the aggregate is read incrementally and only one record-safe raw chunk exists at a time; *rdf2hdt* and *pycottas* each consume it before it is deleted. HDT parts are merged pairwise in a balanced tree with *hdtc* and indexed; COTTAS parts are merged in one streaming *k*-way pass with an *spo* index (other indexes will also be possible in upcoming version). (3) **Validation and packaging:** each artifact is decoded back and its triple count compared with the source; the graph is then checked against the VCF Core SHACL shapes and against thirteen questions answered both by SPARQL and by a VCF parser. HDT and COTTAS files are queryable as written; gzip or Brotli packages are for transport and must be unpacked before querying.

emitted in two profiles. In the *expanded* profile every sample value is its own resource, which makes genotypes directly queryable. In the *condensed* profile values are stored as sample-ordered vectors, reducing the number of RDF resources while requiring decoding for access to individual sample values. Both profiles keep the same file, header and record context.

Sample-profile scaling. Let V be the number of variant records, S the number of samples, and F the number of FORMAT keys per record. Both profiles describe the S samples once per file. For a uniform FORMAT layout, expanded representation adds VS sample-call resources and VSF FORMAT-value resources, whereas condensed representation adds V cohort matrices and VF value vectors. Excluding the record

and header structure common to both profiles, and assuming bounded ploidy and FORMAT-value arity, the sample-layer triple counts therefore scale as

$$\begin{aligned} T_{\text{expanded}} &= \mathcal{O}(S + VSF), \\ T_{\text{condensed}} &= \mathcal{O}(S + VF). \end{aligned} \tag{1}$$

The condensed profile removes the per-sample factor from the repeated RDF structure, but still stores all VSF values inside vector literals, so its size in bytes keeps growing with S even though its triple count does not (Section 3.3). Consequently, a nearly constant triple count as S increases does not imply constant file size or sample-access cost. This trade-off motivates retaining expanded representation for direct sample-level RDF queries and condensed representation for cohort-scale storage, and measuring triple counts, artifact bytes, and peak workspace separately.

The vocabulary itself carries classes, properties, domains and ranges but no OWL restrictions. Value and cardinality constraints are expressed as SHACL shapes, which VCF-RDFizer v3.1.0 bundles from vocabulary commit 5bfd19d and applies during validation (Section 3.5).

Core entities are minted from deterministic IRI templates derived from the source file and record position, so identifiers are reproducible across runs. Section 3.5 confirms this with byte-identical repeated conversions. A simplified view of the core classes is shown in Figure 2, and the full vocabulary, with worked single- and multi-sample examples, is documented at <https://ecrum19.github.io/vcf-core-vocabulary/>.

2.2 Tool Workflow

VCF-RDFizer is implemented as a Python command-line tool that coordinates five stages: (1) conversion of each VCF into records, header-line, and file-metadata TSV logical sources; (2) TSV-to-RDF transformation with modular RML mappings and RMLStreamer; (3) construction of one the materialized N-Triples graph (either plain or space-optimized); (4) optional creation of raw-RDF gzip/Brotli artifacts and/or queryable HDT/COTTAS representations; and (5) native representation validation, packaging, cleanup, and run-metric reporting. The command-line interface separates these concerns through `--rdf-storage-mode`, `--rdf-compression`, `--representations`, and `--artifact-compression`. Thus, an unwrapped `.hdt` or `.cottas` file can be retained for querying while an additional `.gz` or `.br` package is created for transport or archival storage.

Mapping rules are authored in Turtle as modular RML rule sets [29] that mediate the conversion from VCF to RDF. This modularity allows rules to be extended or modified without rewriting the full pipeline. To execute the mappings, we use RMLStreamer [30], version 2.5.0, through its standalone command-line interface.

Aggregate storage modes. In `plain` mode, RMLStreamer outputs are appended to one uncompressed `<sample>.nt` graph and the partitioned RMLStreamer output is deleted after incorporation. In `space-optimized` mode, each line-oriented RMLStreamer output is gzip-compressed, appended as a member of one concatenated-gzip `<sample>.nt.gz` stream, and then deleted. Standard gzip readers expose the

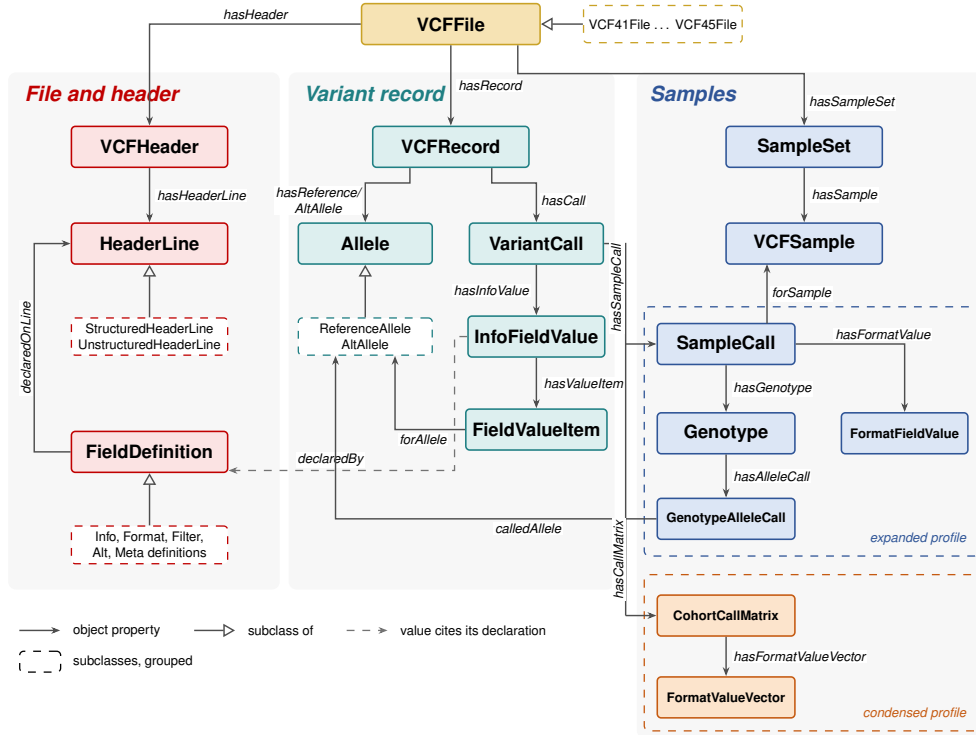
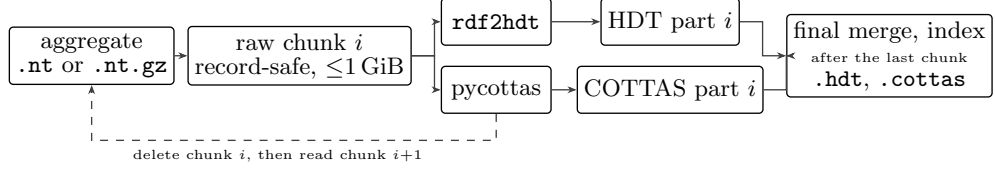


Fig. 2 VCF Core Vocabulary, simplified class diagram. The backbone of the model VCF-RDFizer v3.1.0 emits, drawn from the ontology bundle it ships (vocabulary commit 5bfd19d); all class and property names carry the *vcf:* prefix, <https://w3id.org/vcf-core/vocab#>. A *VCFFile*, typed with its version-specific subclass, links to three layers. In the **header** layer, header lines carry the INFO, FORMAT, FILTER, ALT and META definitions that values cite through *declaredBy*. In the **record** layer, each *VCFRecord* has ordered alleles and a *VariantCall* carrying the INFO values, whose Number=A/R/G items point at the allele they describe. In the **sample** layer, the call links either to per-sample *SampleCalls*, genotypes and allele calls (*expanded* profile) or to one *CohortCallMatrix* of sample-ordered vectors (*condensed* profile). Structural-variant, phasing, local-allele and value-typing classes are omitted; the full vocabulary is documented at <https://ecrum19.github.io/vcf-core-vocabulary/>.

concatenated members as one logical N-Triples sequence. This avoids retaining all RMLStreamer parts and a second full uncompressed aggregate simultaneously.

One-chunk-at-a-time compressed graph representation construction. The current implementation incrementally reads the plain or gzip aggregate, respects newline-terminated N-Triples record boundaries, and materializes one raw chunk in a Docker-managed workspace. Each selected representation builder consumes that chunk before the raw chunk is deleted and the next chunk is created:



Consequently, at most one uncompressed RDF chunk exists at a time. The final HDT/COTTAS parts, merge intermediates, indexes, and converter scratch space still require storage, but the pipeline does not create a full decompressed duplicate of the aggregate.

HDT construction. Each chunk is converted with `rdf2hdt` from `hdt-cpp` [32, 38]. Intermediate HDT files are then merged pairwise in a balanced tree with `hdtc create` [39], which performs the same logical merge as `HDTCat` [40] under a fixed memory budget and without Java; an unpaired file is carried into the next merge round, and each merge’s scratch directory is removed as soon as that merge completes. The final HDT artifact is indexed by `hdtc`, producing the sibling `.hdt.index.v1-1` index without executing a query. Intermediate indexes are not retained because the final merged dictionary and triple layout are authoritative.

COTTAS construction. COTTAS encodes RDF in an Apache Parquet triple table and supports compressed triple-pattern evaluation through columnar pruning and indexes [33]. VCF-RDFizer converts each chunk with `pycottas` [41] and then merges all chunk files in a single streaming k -way merge rather than pairwise, so no global in-memory hash table or full-size temporary spill is built. The merged output carries an `spo` index, which is fixed rather than user-selectable in v3.1.0. (This will change in future version) Each COTTAS operation runs in an isolated temporary working directory so that its local DuckDB scratch database cannot be reused by a later chunk or merge. COTTAS artifacts are queried through a Comunica engine that evaluates SPARQL over the Parquet table with DuckDB [42].

Validation and metrics. Before packaging or raw-RDF cleanup, every base HDT or COTTAS artifact is opened with its native reader (`hdt2rdf`, `pycottas.cottas2rdf`) and decoded as a stream. The decoded triple count is compared with the source count recorded by `RMLStreamer` or the partition planner. No full decoded RDF file is written for this check. A missing decoder, unreadable artifact, or count mismatch fails the representation stage and prevents source cleanup. Chunk conversion, merging, final indexing, validation, and packaging are timed separately, while sample-level metrics report cumulative wall/user/system time, maximum resident set size, output sizes, and cleanup status.

Gzip and Brotli [31] are available for raw RDF and artifact packaging; their configuration values are exercised in the covering-set experiment (Section 3.9).

Chunk sizes. By default a chunk targets 512 MiB of uncompressed N-Triples, bounded below by 128 MiB and above by 1 GiB (`--chunk-target-bytes`, `--chunk-min-bytes`, `--chunk-max-bytes`). For constrained workspaces the documented lower-peak setting is 128 MiB target, 64 MiB minimum and 256 MiB maximum, trading additional chunks and merges for a lower temporary peak. Every experiment

in Section 3 used the defaults except six cells of the memory-feasibility sweep, which used the lower-peak setting (Section 3.9).

Pinned toolchain. VCF-RDFizer v3.1.0 fixes its dependencies inside the container image: RMLStreamer 2.5.0, hdtc 1.1.0, pycottas 1.1.0, DuckDB 1.5.5, cyvcf2 0.34.0, pyshacl 0.30.1, Comunica 5.3.0 for N-Triples, @comunica/query-sparql-hdt 5.0.1, the COTTAS Comunica engine 0.1.0, bcftools from the base distribution, and QLever pinned by image digest. `rd2hdt` and `hdt2rd` are compiled from the hdt-cpp repository at image build time, so their exact revision is fixed by the image digest rather than by a version number; the resolved image for each benchmark run is recorded with its command and metrics (Section 2.3).

The implementation, default RML mappings, and more in-depth documentation about the functionalities and approaches described above are available at <https://github.com/ecrum19/VCF-RDFizer>.

From a user-experience perspective, each tagged VCF-RDFizer version is published across PyPI, Conda-Forge, and DockerHub so users can choose pip-, conda-, or container-based installation with matched versions. The GitHub README serves as the primary user guide, including quick-start commands, end-to-end usage examples, command-line option documentation, and troubleshooting notes. For maintainability, the project follows a release-oriented workflow with versioned changelogs, public issue triage, planned periodic dependency/container updates, and extensive unit-testing within the tool to preserve reproducibility over time.

2.3 Tool Testing

Benchmark objectives and public corpus. The benchmark evaluates conversion coverage, scaling with record and sample counts, storage and representation-build costs, semantic preservation, query execution, and configuration coverage. The ten public VCFs in Table 2 span single-sample genomes, a large phased cohort, and sequence-resolved structural variants. They comprise five Personal Genome Project donations [43], one HG002 precisionFDA/Genome in a Bottle challenge file [44], the 2,504-sample 1000 Genomes Phase 3 chromosome-20 call set on GRCh37 [45], the 32-sample HGSVC2 structural-variant call set on GRCh38 [46], and the HG004 and HG005 Genome in a Bottle v4.2.1 GRCh38 benchmark files [47]. Download commands and canonical filename definitions are maintained in the testing repository: <https://github.com/ecrum19/vcf-rdfizer-testing>.

Input selection and experimental scope. The corpus-breadth experiment uses the first 250,000 records of each input, with complete headers retained, except for HG005_GRCh38, which is processed whole. The expanded-representation arm excludes corpus inputs with more than 1,000 sample columns, so the 2,504-sample 1000 Genomes file is assessed separately through 10,000-record slices. The resulting corpus measurements comprise eight 250,000-record conversions and one whole-file conversion. All use space-optimized storage and build both HDT and COTTAS from the same RDF graph (Figure 5).

Record scaling uses HG005_GRCh38 at 10,000, 100,000, and one million records, with three replicates at each size, followed by one whole-file run. Sample scaling uses 1000G_phase3_chr20 held at 10,000 variant records and varies the sample count from 1

Table 2 Public VCF corpus used in the benchmark. File names are canonical `.vcf.gz` stems; subsequent tables abbreviate the GIAB identifiers. The five NG/NB/6082 files are Personal Genome Project donations. Assembly and sample counts were verified from each file’s header. Sizes are approximate compressed download sizes; the tool processes the uncompressed stream (Section 3.4).

File	Provider	Assembly	Samples	MB (gz)	Evaluation role
NG1N86S6FC	Sequencing.com	GRCh38	1	379	Consumer WGS
NG131FQA1I	Dante Labs	GRCh38	1	224	Consumer WGS
NB72462M	Nebula Genomics	GRCh38	1	341	Consumer WGS
60820188475559	PGP upload	GRCh37	1	325	SNP-only profile
60820188474283	Dante Labs	GRCh37	1	222	Consumer WGS
OG00R_HG002	precisionFDA	GRCh38	1	69	GIAB Truth Challenge v2
1000G_phase3_chr20	1000 Genomes	GRCh37	2,504	327	Phased cohort
HGSVC2	HGSVC2	GRCh38	32	31.5	Multi-sample SVs
HG004_GRCh38	GIAB v4.2.1	GRCh38	1	149	Benchmark genome, chr1–22
HG005_GRCh38	GIAB v4.2.1	GRCh38	1	139	Benchmark genome, chr1–22

to 2,504 under both expanded and condensed representations. Holding variant content fixed isolates the sample-count dependence in Equation 1, while testing both profiles quantifies the cost of exposing individual sample values as RDF resources. Source INFO fields are held fixed when sample columns are selected, so these derived inputs measure structural scaling rather than the consistency of INFO allele summaries with the selected samples. Paired storage-mode runs compare peak workspace and wall time on identical inputs. Separate experiments exercise INFO and header representations, all seven configuration axes, individual operating modes, requested memory ceilings, and the three linking tiers.

Version and difficult-input coverage. Nine public inputs declare VCFv4.2, and 1000G_phase3_chr20 declares VCFv4.1. Dedicated version fixtures extend coverage through VCFv4.5 and include an absent file-format declaration [1]. Eleven additional fixtures exercise CRLF line endings, symbolic and breakend alleles, mixed ploidy, missing and partially missing genotypes, unused header declarations, FILTER states, a header-only file, a sites-only input, and a truncated gzip stream. These small inputs isolate specific format features; production-scale evidence for VCFv4.3–4.5 remains outside the public corpus.

Validation and query measurements. Every constructed HDT and COTTAS artifact is decoded with its native reader and checked against the source triple count. The paired query suite in Table 3 is run on designated validation inputs: the difficult-input fixtures, a 10,000-record multi-sample fixture under both sample representations and all three artifact types, and a 100,000-record slice of HG005_GRCh38. These inputs also support graph-integrity checks, SHACL validation, cross-engine comparisons, and determinism and round-trip tests. A separate mutation experiment measures detection of 113 injected corruptions. The corpus-breadth cells supply native decode and triple-count evidence; the full paired workload is not run on those cells. Retrieval measurements use three replicates and report engine setup separately from query execution and representation construction.

Execution environment and measurement protocol. The tagged benchmark configuration uses VCF-RDFizer v3.1.0 (commit d3b34d5) from ecrum19/vcf-rdfizer:v3.1.0, with

resolved image digest sha256:1904e96dde12ab2e2e70d8ee1267765c-293ab100a8bd1b14d5b009b2bf8e34aa. Image rebuilding is disabled during execution so that all cells in the tagged run use the same container. The per-cell manifests retain the actual commit and image identifiers used for each measurement.

Experiments run on two physically separate, identically specified hosts (**vcf-bench-1** and **vcf-bench-2**): Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic, Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM with swap disabled, a 210 GB SSD, Docker 29.7.2, and Python 3.12.3. Each complete experiment is assigned to one host and executed without concurrent benchmark jobs, keeping comparisons within-host. Both hosts execute an identical calibration workload; its results are reported in Section 3.1.

The harness creates a fresh output directory for each cell and records its command, host, tool commit, container image, exit status, and wall time. Tool reports provide stage timings, peak resident set size, triple counts, artifact sizes, and validation outcomes; workspace monitoring records transient disk use separately from final artifact size. Input sizes used in performance comparisons are uncompressed VCF bytes, while Table 2 reports compressed download sizes. Replicated comparisons report variability or confidence intervals; single-run corpus measurements describe the tested input and configuration without estimating run-to-run variation. The per-cell records, normalized query results, and comparison reports are archived in the testing repository.

2.4 Semantic-equivalence and bioinformatic query workload

Syntax and triple-count checks establish that an output is readable, but they do not establish that biologically relevant VCF content can be recovered from the RDF graph. We therefore evaluate thirteen paired queries by executing SPARQL over the RDF representation and independently deriving the expected results from the source VCF. Table 3 specifies the result compared and the scope of every query: Q1–Q6 summarize records and genotypes, Q7–Q10 check metadata and graph structure, and Q11–Q13 check value preservation through digest summaries. The bioinformatic operations are grounded in standard VCF inspection and quality-control operations exposed by VCFtools, bcftools, and programmatic VCF libraries [6, 48, 49].

Q1–Q4 apply to every file because they use fixed VCF fields represented by the current mapping. Q5–Q6 apply only when the source contains sample columns and a GT FORMAT field; an empty result is accepted only after header preflight confirms that GT is genuinely absent. Each input is validated only against the graph converted from it: results are never pooled across files, so inputs on different reference assemblies (GRCh37 and GRCh38) are never compared with one another. A graph is queried as a single source even when it was written as several RMLStreamer parts or built from several chunks, because those are merged into one aggregate or artifact before validation runs.

The same SPARQL text is executed by four engines: QLever, which is the default; the local-file Comunica package `@comunica/query-sparql-file` for N-Triples [50, 51]; `@comunica/query-sparql-hdt`, which accepts an `hdt@path/to/file.hdt` source [52]; and a Comunica engine that evaluates SPARQL over COTTAS files with DuckDB [42].

Table 3 The thirteen-query equivalence workload. Each SPARQL result is normalized and must equal a result derived independently from the source VCF (Q4’s exact FILTER text via `bcftools query`). Ti/Tv, call rate and allele frequency are derived only after their underlying counts agree. Q11–Q13 count values in up to 256 buckets keyed on the first byte of SHA-256 over the IRI and values, which detects value changes that preserve ordinary counts but is not a collision-free proof of equality. For Q9, Q10, Q12 and Q13 the expected result follows the selected INFO and sample representations.

Query	Result compared, fields and scope
<i>Record level — every input</i>	
Q1 Record density	Records per contig and 1-Mb window (<code>CHROM</code> , <code>POS</code>); checks coordinates and integer positions
Q2 Allele shapes	Records per class — SNV, multiallelic, insertion, deletion, substitution, symbolic/breakend, no ALT (<code>REF</code> , <code>ALT</code>)
Q3 Ti/Tv	Transitions, transversions and biallelic SNVs; single-base A/C/G/T alleles only
Q4 FILTER values	Records per exact FILTER string: <code>PASS</code> , <code>.</code> (not applied) or failed codes, combined codes kept whole
<i>Genotype level — inputs with sample columns and GT</i>	
Q5 Genotypes	Calls per sample and genotype class, including missing, haploid and other-ploidy calls; phased and unphased alike
Q6 Allele counts	Sites per recomputed (AC, AN) pair; single-ALT sites with complete 0/1 haploid or diploid calls
<i>Metadata and graph structure</i>	
Q7 File metadata	Exact <code>fileformat</code> , <code>reference</code> and <code>source</code> strings, with markers for absent declarations
Q8 Header census	<code>##</code> lines per header key (<code>INFO</code> , <code>FORMAT</code> , <code>FILTER</code> , <code>contig</code> , ...)
Q9 Predicate census	Triples per predicate against the counts expected from the VCF
Q10 Class census	Resources per <code>rdf:type</code> against the expected inventory
<i>Value preservation — IRI-bound digests</i>	
Q11 Record digest	Records per bucket over each record’s IRI, <code>CHROM</code> , <code>POS</code> , <code>ID</code> , <code>REF</code> , <code>ALT</code> , <code>QUAL</code> , <code>FILTER</code> and raw <code>INFO</code>
Q12 INFO digest	Structured <code>INFO</code> values per bucket, bound to record and key; empty in raw-only <code>INFO</code> mode
Q13 FORMAT digest	<code>FORMAT</code> values (expanded) or sample-ordered vectors (condensed) per bucket, beyond <code>GT</code>

Each engine can be pointed at any of the three artifacts, and `--validation-engine all` runs all four and records whether they agree. On the VCF side, `cyvcf2` provides the primary programmatic oracle, while `bcftools query` preserves the exact FILTER token needed for Q4 [6, 49].

The primary correctness contract is exact equality of normalized query results: metadata strings for Q7 and integer counts for the remaining queries, including the digest-bucket counts in Q11–Q13. Ratios such as Ti/Tv, call rate, and allele frequency are derived only after their underlying counts agree. The comparator preserves contig and sample identifiers, distinguishes `PASS` from `.`, applies the same rules to phased/unphased and missing genotypes, and emits missing/extra/differing rows for localization. Preflight checks additionally validate N-Triples syntax, one-to-one cardinality of core record properties, position datatypes, source/output hashes, tool versions, and record/triple counts.

Coverage of QUAL, INFO and FORMAT. `QUAL` is emitted for every record as a typed literal, with a missing `QUAL` written as `"^^vcfc:Null"`; it is produced by the tool’s record-detail emitter rather than the RML mapping, because RML cannot switch a literal’s datatype per row. `INFO` is emitted in the structured representation

by default — one typed value resource per record and key, decomposed into per-allele items for Number=A/R/G fields — with the complete source string kept alongside as `vcfc:infoRaw`; `--info-representation raw` keeps the string alone. Q1–Q6 do not inspect QUAL, INFO, or FORMAT fields other than GT. The digest queries Q11–Q13 in Table 3 extend this coverage: Q11 hashes each record’s fields and raw INFO string together with its IRI; Q12 hashes structured INFO values with their IRIs; and Q13 hashes per-sample FORMAT values in the expanded representation or sample-ordered FORMAT vectors in the condensed representation. Each query compares a histogram over the first byte of SHA-256, allowing detection of value changes that ordinary counts miss without requiring a full value listing; agreement of these finite histograms does not guarantee value-by-value equality.

This workload establishes *representation equivalence* for the stated fields: a SPARQL engine and a conventional VCF parser should derive the same deterministic summaries from two representations of the same source. It does not establish that the source calls are biologically correct, normalized, left-aligned, or concordant with a truth set. The GIAB inputs can support a later reference-aware truth-concordance layer, but that requires confident regions and normalization beyond the present semantic-equivalence test.

The query files, `cyvcf2` oracle, normalizer and comparator are committed and were executed 78 times across the campaign; Section 3.5 reports the outcomes. Every validation runs the thirteen-query workload in Table 3, preceded by fourteen graph-integrity preflight queries. The cross-engine experiment includes N-Triples, HDT, and COTTAS under both sample profile representations.

3 Results

The results characterize the benchmark design and input scopes defined in Section 2.3. Conversion, storage, validation, and retrieval measurements are reported separately so that the cost and evidence associated with each stage remain explicit.

3.1 Campaign coverage and reproducibility

On the calibration workload the two benchmark hosts produced the same graph: identical triple sets, a byte-identical COTTAS artifact and HDT index, and N-Triples and HDT files that differ only in line order and in the HDT header’s `dcterms:issued` timestamp. Their speed differed: the conversion phase took 26 s on `vcf-bench-1` and 37 s on `vcf-bench-2`, and validation 156 s and 208 s. Timings are therefore compared only within a host, which is why each experiment runs entirely on one (Section 2.3); the calibration characterizes, rather than eliminates, variation between hosts.

The campaign comprises 143 recorded cells over 96.2 h of measured wall time (Table 4). Every cell records its own argv, host, tool commit and image digest, and the integrated record of all cells is archived with the testing repository. A non-zero exit is classified by what the cell asserted: two cells were required to be refused and were (*refused*), two recorded an exit status without asserting an outcome (*recorded*), and three were skipped by a stated guard. No cell failed.

Table 4 The benchmark campaign, grouped by the question each experiment answers. *Recorded*: a non-zero exit that the cell observed without asserting an outcome; *refused*: the cell required the configuration to be rejected, and it was; *skipped*: a guard refusal with a stated reason. Hosts: *vcf-bench-1* (b1), *vcf-bench-2* (b2). Every cell was produced by the published v3.1.0 image.

Experiment	Cells	Outcome	Host	Hours	Question answered
<i>Conversion, scaling and storage</i>					
Records scaling	10	all OK	b1	10.99	How cost grows with records (§3.3)
Sample representation	30	29 OK, 1 skipped	b2	19.69	How the two profiles grow with samples (§3.3)
Storage mode	8	all OK	b1	12.87	Peak workspace of plain vs space-optimized (§3.3)
Corpus breadth	10	9 OK, 1 skipped	b2	26.90	HDT and COTTAS on ten public inputs (§3.4)
<i>Input and configuration coverage</i>					
Representation axes	20	all OK	b1	4.82	VCF 4.1–4.5; cost of INFO and header representations (§3.2, §3.9)
Difficult inputs	16	13 OK, 2 recorded, 1 skipped	b1	0.10	Eleven awkward VCFs and the linkers (§3.2, §3.7)
Covering set	10	all OK	b1	0.35	Every option value and pair (§3.9)
Modes	7	all OK	b1	0.03	Each operating mode on its own (§3.9)
Feasibility	9	all OK	b1	9.03	Memory ceiling × configuration (§3.9)
<i>Validation and retrieval</i>					
Robustness	9	all OK	b1	0.25	Determinism, round trip, mutation score (§3.5)
Equivalence	8	6 OK, 2 refused	b2	6.84	Four engines, both profiles, three artifacts (§3.10)
Query cost	6	all OK	b1	4.27	SPARQL vs cyvcf2 on identical work (§3.6)
Total	143	136 OK, 2 recorded, 2 refused, 3 skipped		96.16	

3.2 Input and version coverage

Declared VCF versions. Six single-purpose fixtures covering VCFv4.1 through VCFv4.5, plus one file with no `##fileformat` declaration, all converted with exit code 0. The generated RML mapping for each version is byte-identical apart from the asserted file class (`vcfc:VCF41File` through `vcfc:VCF45File`), which locates version awareness in the typed file resource rather than in field-level rules. Field handling is therefore data-driven: the VCFv4.5 fixture’s local-allele `FORMAT` keys (`LAA`, `LAD`) and the VCFv4.4 fixture’s phase-set key (`PS`) are carried through as ordinary `FORMAT` entries, raising the per-record triple yield from 210 (4.4) to 252.5 (4.5) on otherwise matched two-record fixtures. This is preservation, not interpretation: nothing in the graph resolves a local allele index to a global allele, so VCFv4.4/4.5 support should be described as lexical carry-through pending a dedicated local-allele semantics layer. The undeclared-version file also converted, via the documented fallback.

Real cohorts. Nine of ten corpus files converted end to end, with HDT and COTTAS built and decode-checked for each (Figure 5). The tenth, the 2,504-sample 1000 Genomes chromosome-20 call set, was skipped by the benchmark harness rather than refused by the tool: the harness withholds any corpus file with more than 1,000

Table 5 Difficult-input fixtures. Both “converted” and “refused with a clear diagnostic” are acceptable outcomes; a crash or a silently wrong graph is not, and neither occurred. Triple counts are for the complete graph the run produced.

Fixture	Outcome	Triples	Validation	Note
CRLF line endings	converted	262	PASS	No stray <code>\r</code> in the last field’s literal
Symbolic alleles	converted	453	PASS	<code></code> not treated as a base string
Breakend ALT	converted	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	converted	350	PASS	Ploidy 4 then 3 followed the data
Half and haploid calls	converted	446	PASS	Four genotype cases stayed distinct
Missing values	converted	321	PASS	Missing QUAL, FILTER, INFO and FORMAT absent from the graph
FILTER variants	converted	486	PASS	PASS, . and a failed code are three states
Declared FORMAT unused	converted	293	PASS	Header declaration represented although never used
Zero data records	converted	189	PASS	Header-only input; no data records
Sites only	converted; validation not completed	245	incomplete	Graph and HDT artifact produced
Truncated gzip	refused at decompression	—	not reached	Refused in 0.58s with a decompression diagnostic; intended

sample columns from the expanded arm, because the expanded representation emits one call resource per record and sample. The resulting sample-dependent growth in RDF structure is described in Section 2.1. The whole file was therefore not converted in either representation. Its full sample dimension was exercised instead through 10,000-record slices carrying all 2,504 samples, which converted in both representations, and Section 3.3 quantifies why the expanded arm cannot be run on the whole file at this disk budget. The tool also applies a workspace check: before converting, it estimates the expanded workspace from the record and sample counts and refuses the run when that estimate exceeds three quarters of the free space, naming the condensed representation as the alternative. The completed corpus runs cover eight 250,000-record slices and one whole file (HG005_GRCh38, 2.36 GB uncompressed). Together with the 1000 Genomes sample slices, they provide feature coverage across ten public inputs.

Difficult inputs. Eleven adversarial fixtures were run (Table 5; fixture files are named `awkward_*` in the testing repository). Nine converted and passed validation, including CRLF line endings, symbolic alleles, breakend ALT syntax, ploidy 4 and 3 in the same file, half-calls and haploid calls, missing values in every optional field, header keys declared but never used, the three-way `PASS`/. /failed-filter distinction, and a header-only file with no data records. Two cells exited non-zero. One is a refusal to convert: the truncated gzip stream, rejected in 0.58s at the decompression stage with a clear diagnostic, which is the intended behaviour. The sites-only file produced 245 triples and an HDT artifact, but its paired validation was not completed.

3.3 Scaling with records, samples, and storage mode

Records. A four-rung ladder derived from HG005_GRCh38 (10 k, 100 k, 1 M records, then the whole 2.36 GB file) was run in triplicate at the first three rungs (Figure 3a). Triple production is exactly linear in records: the fitted exponent is +0.999 (95% CI [+0.991, +1.007], $R^2 = 1.0000$), and the per-record yield is constant at 170.9–171.9 triples. Final artifact bytes scale at +1.033 and peak workspace at +1.025; end-to-end wall time scales at +1.064 from 10 k to 1 M records (40.7 s, 423.7 s, 5,476.0 s; replicate $sd \leq 11.0$ s). Peak resident memory of the mapping stage is effectively flat at +0.115, staying between 1.0 and 1.7 GB across a hundredfold change in input — the streaming design is memory-bounded, and it is disk, not RAM, that scales with the data. The one exception is the queryable-representation build on the whole 2.36 GB file, which peaked at 4.9 GB. The RMLStreamer mapping stage itself accounts for only 12.2–41.4 s of that time; end-to-end cost is dominated by the downstream queryable-representation build.

Samples. The same 10,000 variant records were re-emitted against a sample ladder of 1, 4, 16, 64, 256, 1024 and 2504 columns, holding variant content fixed (Figure 4a). The two sample representations separate exactly as the semantic model predicts: condensed triple production has a fitted exponent of +0.001 (95% CI [+0.000, +0.002]) — 1,439,503 triples at one sample and 1,452,018 at 2,504, a 0.9% increase over a 2,504-fold change — while expanded production has an exponent of +0.795 (95% CI [+0.649, +0.940]), reaching 627,372,018 triples, 432 $\times$ the condensed graph for the same variants. End-to-end wall time follows: exponent +0.183 condensed versus +0.788 expanded (250 s versus 15,269 s at 2,504 samples), and the stored artifact grows from 67 MB to 8.77 GB. Triple counts understate what the condensed profile stores, because its per-sample values move into vector literals rather than disappearing (Figure 4b). From 1 to 2,504 samples its HDT grows 5.8 $\times$ (10.2 MB to 59.2 MB) while its triples grow 0.9%; its gzip-framed N-Triples grow only 1.25 $\times$ (6.5 MB to 8.1 MB), because sample-ordered genotype vectors compress well. Measured in bytes, the expanded-to-condensed gap at 2,504 samples is 322 $\times$ in gzip-framed N-Triples and 69 $\times$ in HDT, against 432 $\times$ in triples (Figure 4c). The condensed HDT nevertheless stays smaller than the uncompressed input VCF (59.2 MB against 101.6 MB), whereas the expanded HDT is 40 $\times$ larger than it (4.10 GB). Extrapolating the expanded representation to the full 1,812,841-record chromosome-20 call set gives roughly 2 TB against a 189 GB volume, which is the measured basis for the guard described in Section 3.2. Under the tested disk budget, condensed encoding is the feasible option for extending conversion to the full cohort.

Storage mode. The plain and space-optimized storage modes were run paired on two inputs (Figure 3b). They are indistinguishable in output — identical triple counts (17,098,746 and 269,272,677) and the same final artifact size, to within one 4 kB filesystem block — and differ only in transient disk. Peak host workspace falls by 9.19 $\times$ (2.98 GB to 0.325 GB) on the 100 k-record input and by 7.68 $\times$ (43.40 GB to 5.65 GB) on the larger one, at a time cost of +5.2% and +3.2% end to end. A pre-registered two-one-sided-test style equivalence check on the mapping stage at a $\pm 10\%$ margin returns +0.0% with a 90% bootstrap CI of [−1.9%, +0.4%], i.e. equivalent, on the replicated input; the single-replicate larger input is indeterminate by construction.

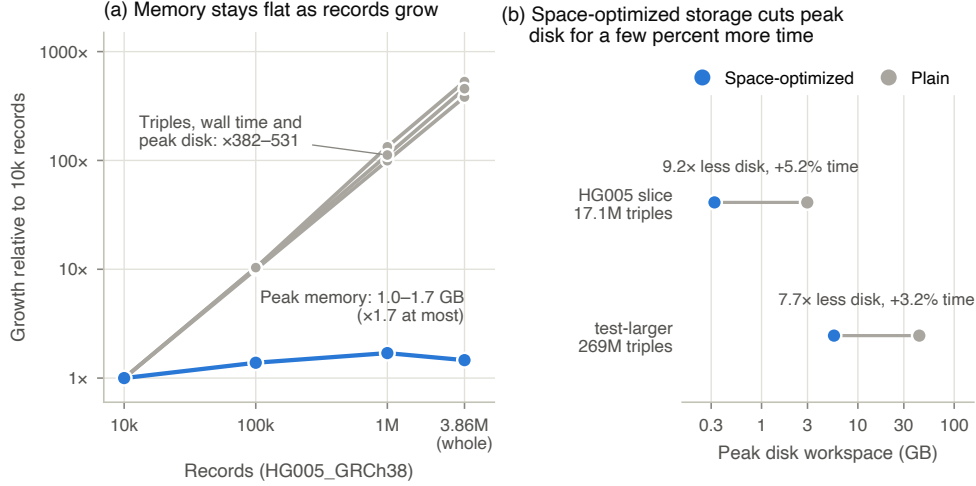


Fig. 3 Record scaling and storage mode. (a) Growth of each measure relative to the 10,000-record rung of the HG005_GRCh38 ladder (medians of three replicates; the whole file is one run). Triples, end-to-end wall time and peak disk workspace grow in proportion to the records, while the mapping stage’s peak resident memory stays between 1.0 and 1.7 GB. (b) Peak disk workspace of the plain and space-optimized storage modes on two inputs, with the end-to-end time cost of space-optimized mode. Final artifacts and triple counts are identical in both modes. Both panels are from the v3.1.0 run on `vcf-bench-1`.

The storage-mode effect is therefore a reduction in peak workspace while preserving final output size and content.

3.4 Constructing queryable representations

Figure 5 compares artifact size and build time across the corpus-breadth experiment. Every corpus cell was run with `--rdf-storage-mode space-optimized`, `--hdt-strategy partitioned` and `--representations hdt,cottas`, so HDT and COTTAS were built from the same graph in the same run.

Two results stand out. First, COTTAS is consistently the smallest artifact: $0.37\text{--}0.55\times$ the size of the stored gzip-framed N-Triples, and $0.27\text{--}0.37\times$ the size of HDT built from the same graph. Second, uncompressed HDT is *larger* than the gzip-framed N-Triples it was built from, by $1.36\text{--}1.82\times$. HDT provides indexed triple-pattern access, but does not reduce size relative to the gzip-framed N-Triples in this experiment. Construction cost runs the other way: COTTAS takes $1.5\text{--}3.0\times$ longer to build than HDT on the same graph (31,230s versus 20,911s on the whole HG005_GRCh38).

Peak workspace is $4.7\text{--}30.3\times$ the uncompressed input across the corpus, with the whole-file cell peaking at 13.76 GB against a 2.36 GB input while producing a 10.88 GB final tree. The whole-file conversion completed in 16.07 h end to end, of which the RMLStreamer mapping stage was 146.8 s.

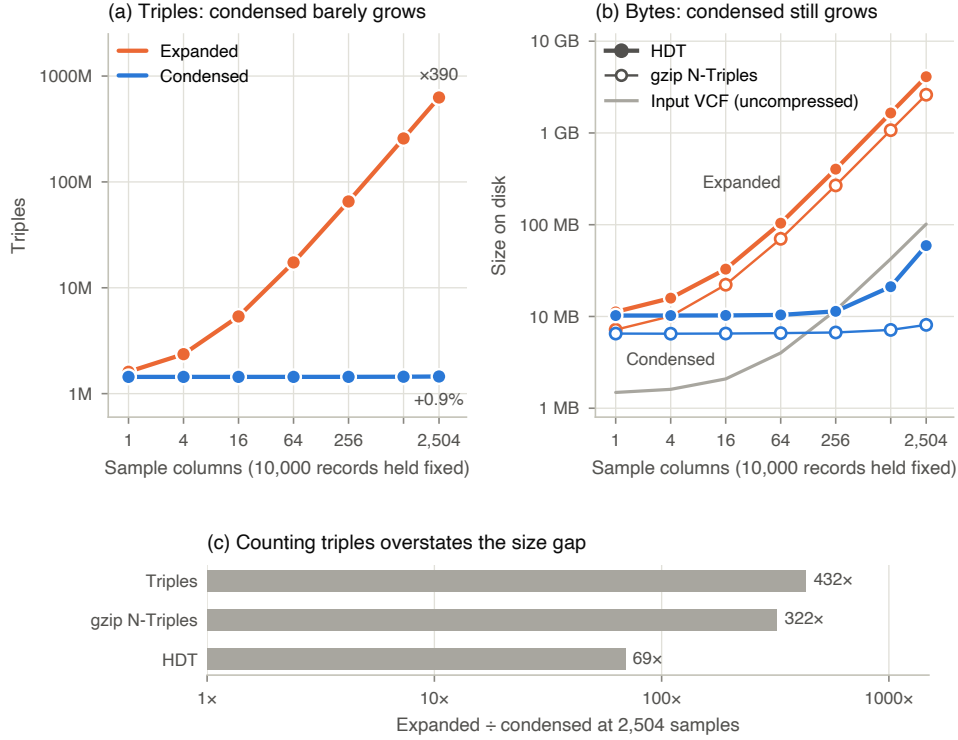


Fig. 4 Triples against bytes for the two sample profiles. The same 10,000 1000G_phase3_chr20 records re-emitted against 1 to 2,504 sample columns (the unlabelled tick is 1,024); medians where the rung was replicated. (a) Triples. The expanded profile adds resources for every sample call; the condensed profile adds five triples per sample and keeps each record’s per-sample values in sample-ordered vector literals. (b) Size on disk of the two stored artifacts, HDT (filled) and the gzip-framed N-Triples that space-optimized mode writes (open), with the uncompressed input VCF for reference. The condensed artifacts grow although their triple count barely changes, because the vectors lengthen with every sample. (c) The expanded-to-condensed ratio at 2,504 samples, measured three ways.

3.5 Semantic validation and graph reproducibility

The tool ships a validation stage that executes SPARQL against the graph it just wrote and compares every answer with an independently computed VCF-side result from `cyvcf2`, with `bcftools` supplying exact FILTER text. Across the campaign this stage ran 78 times over 143 cells, on N-Triples, HDT and COTTAS artifacts, under four SPARQL engines (QLever, Comunica, an HDT-backed engine and the DuckDB-backed COTTAS engine). In v3.1.0 it also applies the vocabulary’s SHACL shapes by default (Section 2.1); the default profile ran on 58 validations and passed all of them with no violations, at 0.8–396.8 s each. The query-cost cells run without shapes, so that their timings measure the queries alone. Table 6 summarizes what that produced. The complete thirteen-query workload is defined in Table 3: Q1–Q6 summarize records and genotypes, Q7–Q10 check metadata and graph structure, and Q11–Q13 compare digest

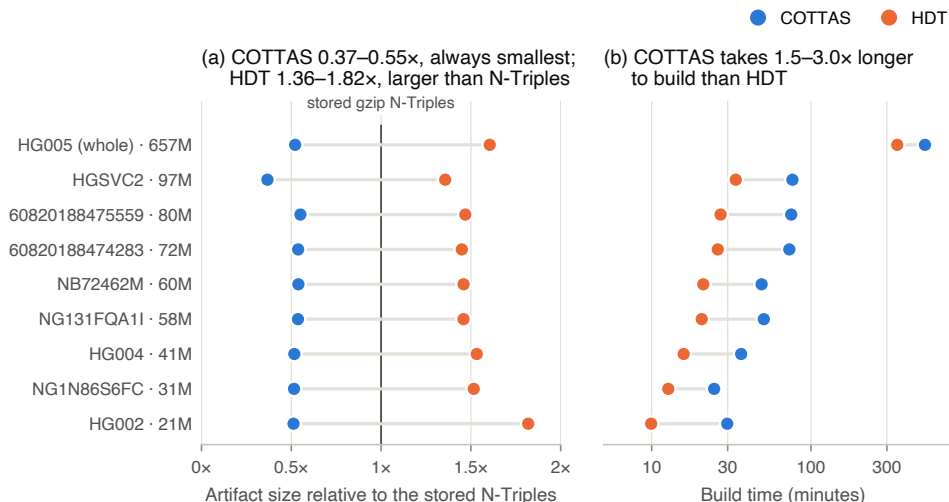


Fig. 5 Queryable representations in the corpus-breadth experiment, one run per completed input, ordered by graph size (triples after each label). Eight files use their first 250,000 records with headers retained; HG005_GRCh38 is processed whole. The 2,504-sample 1000 Genomes cell is guard-skipped in expanded mode (Section 3.3). Dataset labels are abbreviated; full names are in Table 2. (a) Artifact size relative to the stored gzip-framed N-Triples that space-optimized mode writes. (b) Build time of each representation from the same graph, on a log scale. Every artifact passed the tool’s native streamed decode and source-versus-decoded triple-count check.

summaries of record, INFO, and FORMAT values. Fourteen further graph-integrity preflight queries run before them.

Of 988 paired query comparisons, 984 returned exact equality and 4 returned a verified not-applicable: the two genotype questions on each of the zero-record fixture’s two artifacts, since a file with no records has no genotypes. None mismatched. All 1,144 cross-check invariants passed. Engine agreement was recorded as true in all 76 completed validations, including the 25 in which all four engines answered independently; the two sites-only validations were incomplete and contribute no paired-comparison result. The tool’s own native decode check — stream the built representation back and compare source and decoded triple counts — passed 152 times out of 152 across 118 HDT and 34 COTTAS artifacts. Syntactic validation by *rapper* passed on all 76 artifacts it reached.

Three graph-stability properties were tested directly rather than assumed. Two conversions of the same input produced byte-identical N-Triples (SHA-256 661578e7..., 17,098,746 triples, zero duplicate lines). A compress-then-decompress round trip reproduced that same digest exactly. Rebuilding the index over an already-indexed artifact left its digest unchanged.

How much would this validation catch? A mutation-score experiment injected 113 targeted corruptions into the graph — wrong allele, wrong coordinate, wrong filter token, wrong file metadata, blank nodes, flipped phasing, corrupted sample index — and measured how many the validation suite detects. With all three SHACL profiles

Table 6 Validation evidence, grouped by the question each layer answers. Counts are pass/total, derived from archived reports. The sensitivity block is what licenses the two above it: a suite that cannot fail proves nothing by passing.

Check	Passed	What it establishes
<i>Is the graph faithful to the source VCF?</i>		
N-Triples syntax (rapper)	76/76	The serialization parses
Paired SPARQL vs cyvcf2 (Q1–Q13)	984/988 [†]	Exact equality of normalized query results
Cross-check invariants	1,144/1,144	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	76/76	QLever, Comunica, HDT, COTTAS (25 with all four)
Native decode and triple count	152/152	118 HDT, 34 COTTAS; source = decoded
<i>Is the pipeline reproducible?</i>		
Determinism, two runs	1/1	Identical SHA-256; 17,098,746 triples, 0 duplicate lines
Compress and decompress	1/1	Lossless through the packaging path
Index idempotence	1/1	Re-indexing does not perturb the artifact
<i>Would a fault actually be noticed?</i>		
Injected faults caught, queries only	96/113 (0.850)	17 faults, spanning 10 mutation classes, are invisible to the queries
with SHACL shapes enabled	113/113 (1.000)	Shapes catch all 17; both controls still pass
SHACL shapes, default profile	58/58	Cardinality and datatype constraints, 0 violations

[†]Four comparisons were verified as not-applicable rather than passed; they are excluded from paired-comparison totals.

enabled it detected all 113, a score of 1.000. The two layers contribute differently, and the split is the useful part of the result. The comparison queries alone detect 96 (0.850); the other 17 fall into ten classes that the queries structurally cannot see — allele kind and value are counted but never compared against the raw ALT column, header attribute values and the derived contig count are counted but not read, and record index, sample index, called allele and phasing status have no oracle — and the shapes catch every one of them. Both unmutated controls pass in both sample representations, so the shapes are not simply rejecting conformant graphs. The perfect score has a price: the two profiles that check uniqueness and value agreement self-join the graph and cost 33.7s and 73.2s on a 2,000-triple graph, against 1.7s for the default profile, which checks cardinality and datatypes only. v3.1.0 therefore applies the default profile to every validation and reserves the full set for inputs up to 16 MiB.

3.6 Retrieval cost: SPARQL against a VCF parser on identical work

Figure 6 reports the retrieval experiments. The protocol, fixed before the run: three replicates; a cold start per cell; identical question scope on both sides; cyvcf2 as the VCF-side oracle with `bcftools query` for exact FILTER text; engine setup time

reported separately from query time; and representation-build time excluded from both columns and reported instead in Section 3.4. Result equality was verified before any timing was compared — every comparison in Figure 6 is a PASS.

The dominant finding is that the artifact barely matters and the engine matters a great deal. On the 17.1M-triple graph, per-query times against N-Triples, HDT and COTTAS differ by under 5% for the same engine (Figure 6c), because each engine materializes what it needs. Across engines on the 0.96M-triple graph, the same thirteen questions cost a mean of 1.1s under QLever, 23.4s under Comunica, 38.2s under the COTTAS engine and 45.1s under the HDT-backed engine (Figure 6b), with all four engines agreeing in every run. QLever is the outlier, 20–40× faster than the rest, while the other three lie within a factor of two of one another.

The comparison with the VCF parser depends on whether one question or the complete workload is evaluated. For a *single* question against the 17.1M-triple graph, cyvcf2 must scan the VCF and costs 11.5s regardless of which question it is, whereas QLever answers in 0.005–4.62s: SPARQL is 2.5× to 2300× faster per question (Figure 6a). For the *whole thirteen-question batch*, cyvcf2 answers all of them in one 12.4s pass, while QLever needs 17.1s of query time plus 22.8s of one-time setup. The RDF path wins on selective, repeated and ad hoc interrogation; the parser wins on a single exhaustive sweep. These timings exclude the one-time conversion and representation-build costs reported in Section 3.4.

3.7 External linking

The tool exposes external linking as a separate `--mode link` stage with three tiers, and all three were exercised on the same 86-record fixture (Table 7). These measurements characterize execution and provenance capture on the test fixture; they do not establish linking accuracy on a real cohort.

Tier 1, identifier rewriting, works and is cheap: on an 86-record graph the `rsid-dbsnp` linker emitted 86 `linking:sameVariantAs` triples to `identifiers.org` dbSNP IRIs in 2.5ms with no network access, no skipped records and a recorded manifest digest. But tier 1 is by construction a syntactic rewrite of an identifier that is already in the ID column; it demonstrates plumbing and provenance capture, not entity resolution. The fixture’s identifiers are synthetic (`rs00000001–rs00000086`), so all 86 of those links point at dbSNP records that do not exist, and the linker records its assertion as unverified for exactly that reason.

Tier 2, coordinate-interval lookup against a bundled reference, ran successfully and produced *zero* links from the same 86 records. The run is well-formed — it records the reference digest, the GRCh38 assembly, 86 unique keys and no skipped records — but the bundled demonstration gene set does not overlap the fixture’s coordinates. Tier 2 is therefore demonstrated to execute, not demonstrated to link.

Tier 3, live service resolution against Ensembl, ran with network access enabled for that cell only. It batched the 86 keys into one request, which returned HTTP 200 after 20.8s with an empty result (the recorded response digest is that of `{}`), and it emitted no links. That is the correct answer: Ensembl knows none of the fixture’s synthetic identifiers, so where tier 1 linked all 86 of them unchecked, tier 3 correctly linked none. The run records the request hash, response digest, HTTP status and fetch

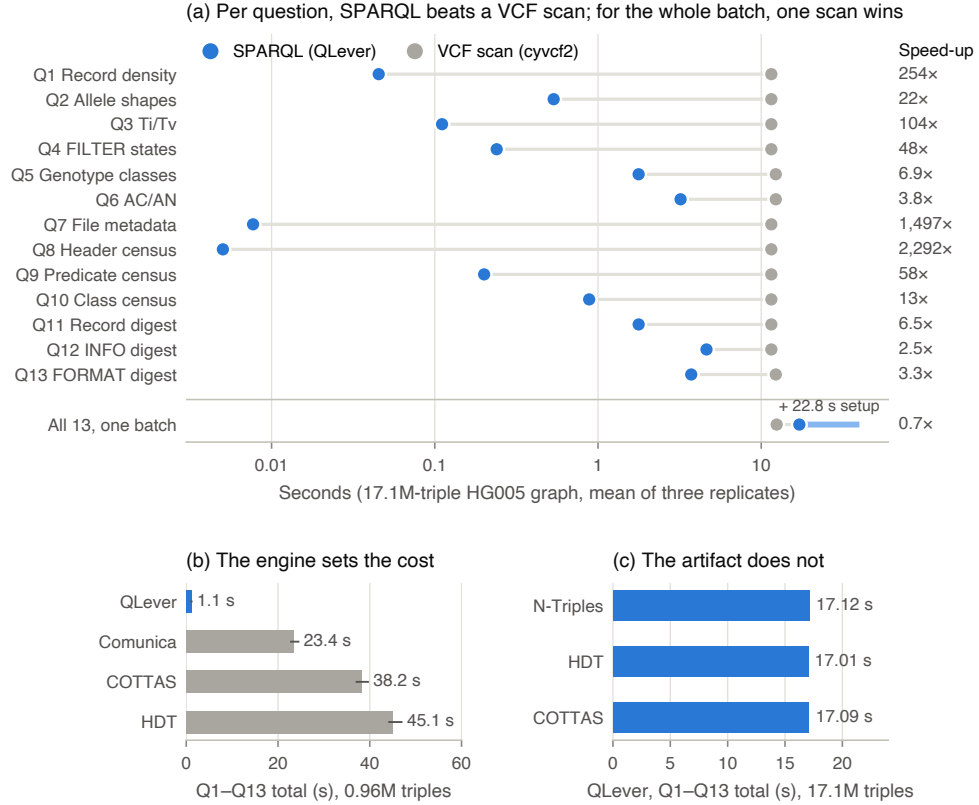


Fig. 6 Retrieval cost. Every comparison returned **PASS** before any timing was compared, and representation-build cost is excluded throughout (Section 3.4). (a) Paired per-question cost on the 17.1M-triple HG005.GRCh38 graph (100,000 records), mean of three replicates. The cyvcf2 cost is that of answering the one question from the VCF, which requires a scan whichever question it is. QLever’s one-time setup is excluded from the per-question rows and drawn on the batch row, where one parser pass answers all thirteen questions. (b) The same thirteen questions under each engine on the 0.96M-triple graph: mean of nine runs (three artifacts in each of three replicates), with the range. All four engines agreed in every run. One-time setup, not included: QLever 2.0–2.1 s, Comunica (@comunica/query-sparql-file) 11.7–12.5 s, the DuckDB-backed COTTAS engine 6.6–39.2 s, and the HDT-backed Comunica engine (@comunica/query-sparql-hdt) 6.1–11.3 s. (c) QLever on the 17.1M-triple graph from each artifact, mean of three replicates; setup 22.7–23.0 s in each case.

time, so the result can be audited, although, as a live service, it is not reproducible offline. The custom-mapping path was skipped for want of a supplied rules file.

The fair summary is that the linking architecture — tiered linkers, offline and live operation, per-linker manifest, reference and response digests, link counts and timings in the run record — is implemented, instrumented and runs end to end, and that its real-world yield is untested because the fixture contains no real identifiers. A credible demonstration needs a real annotated cohort against a real dbSNP or Ensembl release

Table 7 Linking tiers as exercised. Input is the same 86-record graph in every case.

Tier	Linker	Keys	Links	Time (s)	Network	Status and limitation
1	rsid-dbsnp	86	86	0.0025	none	Works; an identifier rewrite, not entity resolution; the synthetic rsIDs are linked unchecked
2	gene-demo	86	0	0.0012	none	Executes and records the reference digest; the bundled demo interval set does not overlap the fixture
3	rsid-ensembl	86	0	20.8	1 request	Runs against live Ensembl (HTTP 200, empty result, digest recorded); correctly finds none of the synthetic rsIDs

with recall and precision reported against a known overlap, and that experiment has not been run.

3.8 Policy attachment demonstrator

VCF-RDFizer also includes a demonstrator plug-in, `vcf-rdfizer-policy` (v0.1.0), that attaches ODRL policies [8] to converted graphs. It postdates v3.1.0 and is not part of the benchmark campaign. It runs on a synthetic cohort of five single-sample VCFs: 139 records at real GRCh38 loci, with synthetic genotypes, converted by v3.1.0 in both sample profiles. Each participant’s consent is a policy on their file. Two cohort-wide rules target a genomic region (*BRCA1*, readable only for clinical care) and a single variant (the APOE ϵ 4-defining rs429358, readable only for disease-specific research). Purposes are GA4GH Data Use Ontology terms [53], matched through the DUO hierarchy, so a disease-specific study falls within a general-research consent but not the reverse (Figure 7a). Conflicts resolve deny-wins, and anything no permission covers is withheld.

Evaluated for three requesters, one set of policies yields three release views (Figure 7b), with 37, 76 and 56 of the 139 records released. Each view differs in which consents admit which files: P003’s health-research consent admits the disease-specific study but not a general-research request. It also differs in which cohort rule is lifted: *BRCA1* for clinical genetics only, and rs429358 for the disease-specific study only. P004 withdrew, and that prohibition overrides every permission. Each view is checked against an oracle computed directly from the VCF text, without the graph or SPARQL, and must equal it exactly: an extra record is a leak, and a missing one is over-withholding. All three views pass in both sample profiles, and each of six planted faults is detected: a reinstated *BRCA1* record, a restored triple of the withdrawn file, a deleted record, a dangling reference, a swapped view, and a changed policy. The check also caught a real defect during development. Every record decision was correct, but a withdrawn file’s header and sample set remained in every view, which the record counts alone could not show.

Because the policies are RDF in the same graph as the data, one SPARQL query can ask which records a collaborator may not receive, and under which rule. A VCF

has no place to state a region- or variant-level policy, and repository consent codes apply to whole datasets. The demonstrator’s limits are deliberate and stated in its documentation:

- the cohort is a small synthetic fixture;
- evaluation happens in memory, after conversion;
- views keep the original IRIs, since there is no pseudonymization;
- obligations are recorded but not enforced.

It provides governed release, not anonymization: a released genotype remains identifying.

3.9 Configuration coverage, operating modes and memory feasibility

A ten-row covering set exercised every value of all seven configuration axes — sample representation, INFO representation, storage mode, RDF compression, representations, artifact compression and HDT strategy — and all ten rows exited 0 and passed validation, including the shapes. Pairwise coverage is complete except for three combinations the wrapper structurally refuses (`--hdt-strategy single` beside space-optimized storage, beside COTTAS, and beside `hdt,cottas`), so the remaining gaps are specification, not omission. Each of the tool’s operating modes — `tsv`, `compress`, `decompress`, `index` for HDT and for COTTAS, a conversion-only phase and a validation-only phase — ran independently and exited 0, confirming that conversion and validation are separable stages rather than one monolithic command.

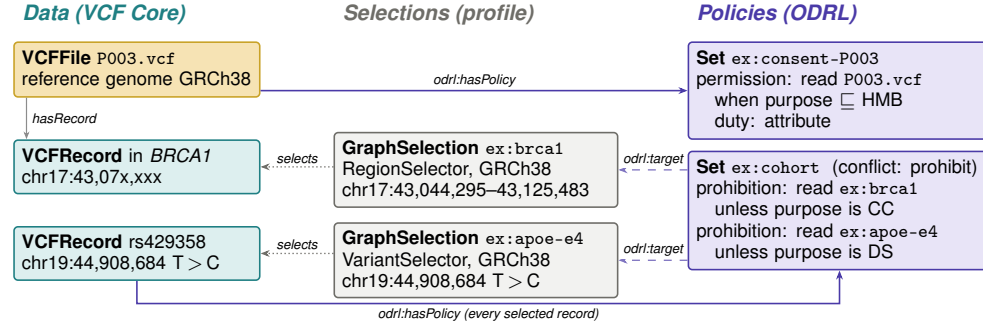
A feasibility sweep crossed three memory ceilings (8, 16 and 31 GB) with three configurations on a one-million-record input. All nine cells completed; peak resident memory was 1.34–1.93 GB in every case, below every requested ceiling, which is consistent with the flat memory scaling in Section 3.3. This experiment carries one caveat that must be reported with it: the ceiling was requested through Docker memory limits but the harness could not confirm that the pinned release forwards them to its container invocations, so these cells establish that the workload is memory-light, not that it survives a genuinely enforced cap.

Finally, the INFO representation axis has a measurable cost: on the structural-variant file, structured INFO produced 96.75M triples against 89.01M for raw INFO (+8.7%) and a 7.4% larger artifact, and on the 100,000-record HG005_GRCh38 input it produced 17.10M against 12.52M (+36.6%). Raw INFO still carries the allele layer here, because the expanded sample profile references it (Section 3.11); before that fix the raw arm omitted it and the difference looked larger than it is. Structured header representation costs almost nothing by comparison (17,098,746 against 17,094,293 triples, +0.03%).

3.10 Cross-encoding equivalence and remaining limitations

Both sample encodings are equivalent across every engine and artifact. The equivalence experiment converted one 10,000-record, multi-sample fixture in both sample representations and both storage modes, built HDT and COTTAS from each,

(a) Where policies attach



(b) What each requester receives

	General research purpose GRU	Alzheimer's study purpose DS	Clinical genetics purpose CC
P001 (GRU + CC)	21 records	22 records	33 records
P002 (GRU + CC)	16 records	16 records	23 records
P003 (HMB)	no consent	18 records	no consent
P004 (withdrew)	withdrawn	withdrawn	withdrawn
P005 (DS)	no consent	20 records	no consent
BRCA1 (region: CC only)	0 of 19	0 of 37	19 of 19
rs429358 (variant: DS only)	0 of 1	3 of 3	0 of 1
	37 of 139 records released 7,869 triples withheld	76 of 139 records released 4,649 triples withheld	56 of 139 records released 6,539 triples withheld

Fig. 7 Policy attachment demonstrator (vcf-rdfizer-policy v0.1.0, after the v3.1.0 release).

(a) Participant P003's consent is attached to its file. The cohort policy's two prohibitions target a region selection (*BRCA1*) and a variant selection (APOE ϵ 4, rs429358). **attach** writes the **odrl:hasPolicy** links, so policies and data can be queried together. GRU, HMB, DS and CC are the DUO terms for general research use, health/medical/biomedical research, disease-specific research and clinical care use; $\sqsubseteq$ is DUO subsumption. (b) What three requesters receive from the same policies over the synthetic five-file cohort. Blue is released and gray is withheld. The two cohort-rule rows count records in the files the requester may otherwise see. Every view equals an oracle computed from the source VCFs.

and ran the full twenty-seven-query suite (the fourteen preflight checks and Q1–Q13) under all four engines against all three artifacts. All sixteen validations passed with all four engines in agreement, on graphs of 0.96M triples (expanded) and 0.79M triples (condensed). The complete condensed cell took 70.4min end to end. Two further cells asserted that the tool must refuse **--hdt-strategy single** beside COTTAS and beside space-optimized storage, and it did, with exit status 2. The tested workload

therefore supports representation equivalence across both sample encodings, all three artifacts, and all four engines.

Scope limits. The sample ladders hold INFO fields fixed while changing sample columns, so they characterize structural scaling rather than INFO-dependent cohort summaries. Corpus conversion covers eight 250,000-record slices and one whole genome; the 2,504-sample chromosome-20 call set is exercised only through 10,000-record slices. The full paired query workload is run on fixtures and a 100,000-record HG005_GRCh38 slice. Correctness evidence for the corpus-breadth cells is limited to native decoding and source-versus-decoded triple counts. These scopes support comparisons within the measured workloads, without establishing whole-cohort throughput or exhaustive value-level validation for every public input.

3.11 Issues identified during testing and their resolution

Testing identified problems in measurement, conversion, and validation. This subsection brings together the historical benchmark context, the observed failures, and the corresponding corrections. The performance and semantic-equivalence results elsewhere in this paper describe the current campaign and its stated input scopes.

Measurement and reporting in the legacy benchmark. The earlier six-file, batch-oriented benchmark exposed two measurement problems. Partitioned compression wall times were under-reported; the timing implementation was corrected in commit [fb48e50](#), and the affected historical durations were reconstructed from wrapper logs and execution timestamps. In addition, an input-size column labelled `.vcf.gz` actually contained the uncompressed VCF stream size. For example, the retained `NG131FQA1I.vcf` contains 1,215,509,730 bytes, matching the reported 1215.5 MB. Performance tables now distinguish compressed download sizes from uncompressed processed inputs.

The compression denominator also affected interpretation. The legacy HDT reduction of approximately 92–95% was measured against uncompressed N-Triples. In the current corpus experiment, HDT is 1.36–1.82 times the size of the stored gzip-framed N-Triples. These observations use different reference artifacts and do not constitute a like-for-like change in compression performance. The legacy conversion and compression tables are therefore excluded from the current performance analysis. The storage evaluation instead pairs the two supported aggregate modes on identical inputs and measures peak workspace separately from final artifact size.

Header-only input omitted its declared samples. A VCF with a declared sample column but no data records produced a graph without the `vcfc:SampleSet` and `vcfc:VCFSample` resources declared by its header. The emitters derived the file IRI from a source name learned from the first data row. The predicate and class censuses (Q9 and Q10) detected the missing resources. Emitting the sample set from the header increased the graph from 181 to 189 triples and allowed validation to complete. The same test exposed an incorrect oracle expectation: `preflight.sample.gt.inventory` expected one `vcfc:sampleId`, although that property belongs to a per-record `vcfc:SampleCall`. The oracle was adjusted to expect no such calls when the file contains no records.

COTTAS query execution and timeout enforcement. The initial COTTAS query implementation used pycottas’ RDFLib store. On the condensed representation it completed the fourteen preflight queries and Q1–Q4, then failed to complete `q05_sample_genotype_counts`. Two attempts ran for approximately 41 h and 10 h at roughly 190% CPU before being stopped. The second had a 1,800 s per-query timeout and a 14,400 s validation budget, but neither interrupted the in-process query. Both runs are archived. Replacing this path with the DuckDB-backed Comunica engine reduced Q5 to 2–5 s and the thirteen-query workload on the 0.96M-triple comparison graph to 38 s, against 1,402 s through RDFLib (Section 3.6). Query execution now occurs where the timeout can terminate it. The completed cross-engine measurements use this engine.

SHACL report interpretation. Integrating the vocabulary’s SHACL shapes exposed two reporting errors. The validator treated pyshacl’s `conforms` flag as a blocking verdict even for recommendations, causing conformant INFO declarations without `Source` to be rejected. Its violation matcher also expected a report format that pyshacl did not emit. The parser was corrected to classify results by severity, with only `sh:Violation` blocking a run.

Allele references without corresponding resources. Applying the shapes to the configuration covering set identified three failing rows, all combining raw INFO with expanded samples. Allele resources were emitted only for structured INFO, but expanded genotypes referenced them through `vcfc:calledAllele` regardless of the INFO setting. A 1,000-record test produced 1,972 such references without the corresponding allele descriptions. The comparison queries counted those edges without following them, whereas the shapes reported 5,916 violations.

The emitter was changed to produce the allele layer when either representation requires it. The census oracle also needed the same dependency rule: its initial expectation reproduced the emitter’s assumption and rejected the added triples. After both were updated, all ten covering-set rows passed. Each affected row gained 10,846 triples, while the other seven outputs remained byte-identical. Regression checks cover all four INFO/sample representation combinations and require every referenced allele to be described.

Missing-token preflight false positives. The preflight reports contain 608 checks: 599 passed and nine flagged the missing-token rule, all on the 100,000-record HG005_GRCh38 input. The rule looks for bare `".` literals that should have been typed as `vcfc:Null`. It first returned 20 rows there, all on `vcfc:fieldNumber` or `vcfc:genotypeString`, where a dot is legitimate: `Number=.` denotes variable cardinality, and a fully missing genotype is written `..`. The rule was narrowed to exclude those two predicates. In v3.1.0 it still returns ten rows, and they are the same case one layer over: the header declares ten INFO fields with `Number=.`, and the header-attribute layer also carries each declaration’s `Number` attribute verbatim as a `vcfc:attributeValue` of `".`. A follow-up change excludes that attribute’s value as well while still reporting a bare dot on any other attribute; it postdates v3.1.0, so the ten flags remain in this campaign’s reports. None of the flags reflects malformed output.

Sites-only fixture limitation. The sites-only fixture produced 245 triples and a valid HDT artifact, but its two validations ended with `EXECUTION_FAILED`. The fixture declared a `FORMAT` column without sample columns, which `bcftools` rejects. This is a fixture/oracle limitation rather than evidence that the generated graph failed semantic comparison. No completed paired result is claimed for this input, and a conformant sites-only fixture remains necessary to establish that coverage.

4 Discussion

The current benchmark demonstrates conversion across heterogeneous public VCF inputs, with eight 250,000-record slices and one whole 2.36 GB genome processed through RDF, HDT, and COTTAS construction. The whole-file output contains 657.43 million triples and completes in 16.07 h. On the replicated record ladder, peak resident memory remains between 1.0 and 1.7 GB over a hundredfold increase in records, while queryable-representation construction dominates end-to-end time. These results support a memory-bounded workflow whose practical resource requirements are governed primarily by build time and disk space.

The paired storage-mode experiment quantifies the temporary disk requirement directly. Plain and space-optimized modes produce identical triples and final artifacts, while space-optimized mode reduces peak host workspace by 7.7–9.2 times at a 3–5% increase in end-to-end time (Section 3.3). This trade-off is useful when temporary storage limits conversion capacity. Final artifact size alone would not capture it.

Sample representation has a larger effect on graph size. Holding 10,000 records fixed, condensed triple production increases by 0.9% from 1 to 2,504 samples, whereas the expanded graph reaches 432 times the condensed graph at 2,504 samples. Counted in bytes the gap is smaller, 69 times in HDT, because the condensed vectors still carry every sample value; the condensed HDT nonetheless remains smaller than the input VCF. The expanded representation offers individual sample-call resources, but its projected workspace for the whole chromosome-20 cohort exceeds the tested disk budget. The condensed representation is therefore the practical setting for extending the measured workload to cohort scale. Whole-cohort feasibility still requires a dedicated run, because the campaign evaluates that input through 10,000-record slices.

Table 8 summarizes the implementation and evaluation status of VCF-RDFizer v3.1.0. Each row states what the release does and what the campaign measured, so that support and evidence are not conflated.

4.1 Semantic Representation of VCF Challenges

Because VCF encodes both biological observations and file-structure semantics, representation in RDF requires a vocabulary that explicitly models the structural elements of the format itself. Existing ontologies capture important aspects of genomic variation, including FALDO for genomic coordinates, the Sequence Ontology (SO) [28] for variant types, GENO for genotype modeling [54], and HERO for clinically oriented genomic semantics [36], but none fully represent the structural semantics of the VCF specification.

VCF-RDFizer addresses this gap by targeting the VCF Core Vocabulary (<https://w3id.org/vcf-core/vocab#>) [34], a version-aware semantic layer aligned with the structural components of the VCF specification while remaining interoperable with existing genomic ontologies. Keeping the vocabulary a separate, versioned artifact means the converter and the model can evolve and be evaluated independently, and the SHACL shapes the vocabulary publishes become the converter’s own conformance test.

The benchmark supports a division of responsibility between query-based comparison and graph conformance. The paired queries test whether VCF-derived summaries, metadata, and digest histograms agree with the RDF results; SHACL checks cardinality, datatypes, and structural relationships. In the controlled mutation experiment, the queries detect 96 of 113 injected corruptions and the combined validation detects all 113 (Section 3.5). This evidence is specific to the tested mutations and input sizes. Agreement with the VCF parser assesses preservation of represented content; it does not establish the biological correctness of source calls.

4.2 The Published VCF-RDFizer Software Tool

Transformation Architecture and Performance. RML provides a declarative framework in which semantic intent is separated from procedural parsing logic, allowing transformation rules to be versioned and reused across datasets. The mapping stage is not the bottleneck: in the current campaign RMLStreamer accounted for 12.2–41.4s of a 41s to 5,476s run on the records ladder and 146.8s of the 16.07h whole-file conversion, with peak resident memory flat at 1.0–1.7GB across a hundredfold change in input (Section 3.3). End-to-end cost is dominated by building the queryable representations.

The pipeline includes a lightweight preprocessing step that converts VCF files into tabular intermediates compatible with existing RML engines. Because VCF is not yet supported as a native logical source within RML, this step enables stable field extraction and declarative mapping execution. Future extensions to the RML ecosystem could support VCF as a native logical source and eliminate the preprocessing stage.

The paired SPARQL/VCF-parser workload supplies the field-level evidence that syntax checks cannot: 984 of 988 comparisons returned exact equality and the other four were verified not-applicable (Section 3.5). The substantial expansion in raw RDF size remains an operational limitation and motivates both archival compression and directly queryable compressed representations.

Compression Strategies for Semantic Genomic Data. Gzip and Brotli provide packaging options for transfer and archival storage. The current covering-set experiment exercises the supported values of both compression axes and validates their outputs (Section 3.9); it does not provide a dedicated compression-ratio or throughput comparison between the two codecs. Packaged artifacts must be unpacked before direct querying.

For compressed RDF that remains queryable, VCF-RDFizer offers both HDT and COTTAS. HDT combines a dictionary and compressed triple structure with indexed triple-pattern access [32]. COTTAS stores RDF as a Parquet triple table and uses columnar organization and block-range indexes to evaluate patterns over compressed

data [33]; VCF-RDFizer builds it with pycottas and queries it with a DuckDB-backed Comunica engine [42]. These formats provide complementary design points, and the COTTAS paper’s general RDF benchmarks must not be assumed to predict VCF-RDFizer-specific size or query performance.

Representation construction operates on bounded, record-safe chunks. HDT parts are merged pairwise with `hdtrc` under a fixed memory budget [39, 40], while COTTAS parts are combined in one streaming k -way merge. Final representations are indexed and checked through native streamed decoding and source-versus-decoded triple counts. Temporary representation parts, merge workspaces, indexes, and converter scratch data still contribute to peak disk use.

The corpus measurements show a consistent storage/build-time trade-off (Figure 5). COTTAS occupies 0.37–0.55 times the stored gzip-framed N-Triples and 0.27–0.37 times the HDT built from the same graph. HDT occupies 1.36–1.82 times the stored N-Triples, but builds faster: COTTAS requires 1.5–3.0 times its construction time. Artifact choice has little effect on query time for a fixed engine in the tested workload (Section 3.6). These results make storage size and construction cost the principal measured trade-offs between HDT and COTTAS.

Tool Availability and Community Impact. Following best practices for research software publication [55], VCF-RDFizer is distributed as open-source software with public version control and synchronized releases across GitHub, PyPI, Conda-Forge, and DockerHub under the MIT License. Public issue tracking, release notes, and a documented maintenance plan support transparency and long-term sustainability.

Queryable Genomic RDF as a Complementary Access Layer. The selected workload illustrates the practical scope of SPARQL even before external annotations are linked. Density windows, allele-shape summaries, Ti/Tv, FILTER distributions, per-sample missingness/genotype classes, and genotype-derived AC/AN are recognizable bioinformatic operations rather than synthetic graph queries. Reproducing them from RDF tests whether genomic coordinates, alleles, filter semantics, sample identities, FORMAT values, and record-to-sample relationships survive semantic lifting.

The longer-term value of SPARQL is not limited to reproducing operations that VCF-native tools already perform well. Once variant records use stable IRIs and ontology-aligned predicates, a query can combine variant/genotype constraints with linked gene, phenotype, pharmacogenomic, provenance, or policy statements without exporting each relation into a new ad hoc table. Multiple VCF-derived graphs can be queried through the same data model, and Comunica’s source-oriented architecture can query one or more local HDT files through a common SPARQL interface [50, 52]. COTTAS provides another compressed query path through the same Comunica architecture [33, 42]. These capabilities could support cohort QC across independently produced VCFs, integrated variant prioritization, knowledge-graph enrichment, and governance-aware retrieval using standards such as ODRL [8]. Section 3.8 demonstrates the attachment side of this on a synthetic cohort: consents on files, and rules on a region and a variant, evaluated per request and verified against the source VCFs.

VCF-native tools remain an essential comparator and, for many format-specific operations, are likely to remain the operational baseline. `bcftools` and `cyvcf2` operate

directly over VCF/BCF and expose highly optimized parsing, indexing, and genotype access [6, 49]. The RDF pathway adds conversion and representation-build costs in exchange for a generic graph query model and semantic joins. It should therefore be positioned as complementary infrastructure rather than an unconditional replacement for VCF/BCF.

A fair comparison must separate one-time conversion/index construction from repeated query execution; apply identical scopes and allele/genotype rules; record cold-cache conditions, result cardinality, wall time and repetitions; and verify result equality before comparing speed. Section 3.6 follows that protocol, and the answer is conditional rather than directional. On a 17.1M-triple graph, a *single* question costs cyvcf2 11.5s because it must scan the VCF whichever question it is, while QLever answers in 0.005–4.62s — between $2.5\times$ and $2300\times$ faster. For the *whole* thirteen-question batch, cyvcf2 answers all of them in one 12.4s pass against 17.1s of query time plus 22.8s of one-time setup for QLever. The RDF path therefore wins on selective, repeated and ad hoc interrogation and loses on a single exhaustive sweep, and neither reading holds without the other. The artifact makes almost no difference — N-Triples, HDT and COTTAS are within 5% for the same engine — whereas the engine does: on the same 0.96M-triple graph the thirteen questions take 1.1s under QLever and 23–45s under the other three engines. Engine selection, not representation selection, is the performance decision, which is why QLever is the default. None of these figures includes the conversion cost in Section 3.4, which the graph must pay once before any of it applies.

[AUTHOR QUERY: Placeholder: an indexed regional-access arm (tabix/CSI) for the VCF side of the retrieval comparison is under consideration and has not been run. The proposal and its motivation are in tool-docs/proposal-indexed-regional-access-arm.md; keep or remove this paragraph once it is decided.]

4.3 Impact and Future Work

This work demonstrates that VCF datasets can be systematically transformed into RDF using declarative RML semantic mappings. More importantly, we show that traditionally static file-based genomic variant data can be reliably converted to structured knowledge graphs, capable of linking, semantic querying, and governance-aware data sharing.

Three priorities follow from the current measurements. First, validation should scale beyond the fixture and 100,000-record workloads. The two SHACL profiles that check uniqueness and value agreement require substantially more time than the default profile, motivating streaming or indexed implementations of those checks before applying the full suite to large cohorts. Second, conversion and validation should extend from slices to whole multi-sample inputs, including the 2,504-sample chromosome-20 call set in condensed form. Third, linking requires evaluation against a released dbSNP or Ensembl resource with known overlaps and explicit precision and recall. The current experiment shows all three tiers executing, including a live Ensembl query, but on a fixture whose identifiers are synthetic and whose coordinates miss the demonstration reference, so none of them has yet been measured against real overlaps.

Subsequent work will extend semantic coverage of heterogeneous INFO, FORMAT, genotype, header, ordered alternate-allele, and structural-variant constructs; evaluate VCF as a native RML logical source; and develop richer links to clinical and biomedical resources. A separate GIAB truth-concordance experiment could test biological call accuracy, but it must use the appropriate benchmark regions and reference-aware normalization and must remain distinct from representation equivalence.

Ultimately, VCF-RDFizer represents a step toward semantically interoperable genomic data infrastructure in which variant datasets can be compressed, linked, queried, validated, and governed as part of a broader biomedical knowledge-graph ecosystem.

Table 8 VCF-RDFizer capability status and current benchmark scope. “Supported” describes the release; the evaluation column states what the campaign in Section 3 measured, so that corpus inclusion is not mistaken for a completed result.

VCF component / capability	In v3.1.0	Evaluation	Notes / limitations
Core record fields (CHROM, POS, ID, REF, ALT, FILTER) QUAL	Supported	Validated	Q1–Q4 exact-equal against cyvcf2/bcftools in every completed validation (Section 3.5)
INFO	Supported	Validated by the Q11 record digest	Emitted for every record as a typed literal, with a missing QUAL as <code>vcfc:Null</code> ; produced by the record-detail emitter rather than RML
FORMAT and sample/genotype values	Supported; structured by default	Validated by the Q11 and Q12 digests	Typed value per record and key, per-allele items for Number=A/R/G, plus the complete <code>infoRaw</code> string; <code>--info-representation raw</code> keeps the string only
HEADER metadata	Supported	Validated for Q5–Q6; Q13 digest passes	Exact-equal on all applicable artifacts; half-calls, haploid calls and ploidy 4/3 survive conversion (Table 5); called allele, sample index and phasing status are checked by the shapes (Section 3.5)
Multi-sample conversion	Supported	Census queries and shapes	Header lines are typed and their declarations represented; header attribute values are checked by the shapes rather than by a comparison query
Structural and symbolic/sequence alleles	Implemented in model and pipeline	Measured, 1–2504 samples	Condensed is flat in sample count; expanded grows at +0.795 and is not tractable at 2,504 samples (Section 3.3)
Queryable HDT output	Lexically supported; semantic typing partial	HGSVC2 converted; Q2 passes	Symbolic and breakend ALT fixtures convert and validate; ALT remains the source lexical field; detecting the injected allele-value mutations requires the SHACL layer
Queryable COTTAS output	Supported	Measured	118 artifacts, all native decode/count checks passed; 1.36–1.82× the stored N-Triples (Figure 5)
Gzip/Brotli artifact packaging	Supported; <code>spo</code> index fixed	Measured	34 artifacts, all decode checks passed, smallest artifact throughout; queried with the DuckDB-backed engine in both sample encodings (Section 3.10)
Provenance/policy annotations	Supported	Exercised in the covering set	All values of both compression axes ran and validated (Section 3.9); packaged files must be unpacked before direct querying
External linking (tiers 1–3)	Not in v3.1.0; v0.1.0 demonstrator after the release	Exercised on a synthetic fixture	ODRL policies on files, regions and variants with DUO purposes; per-request views verified against a VCF-side oracle; post-conversion only, no pseudonymization; governed release, not anonymization (Section 3.8)
	Implemented, instrumented	Partially exercised	Tier 1 links 86/86 records offline; tier 2 executes but yields zero links on the demonstration reference; tier 3 runs against live Ensembl and correctly links none of the synthetic identifiers (Section 3.7)

Declarations

4.4 Conflicts of interest

None declared.

4.5 Funding

Elias Crum acknowledges support from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Bart Buelens acknowledges support from ... Ruben Taelman acknowledges support from ... Gokhan Ertaylan acknowledges support from ...

4.6 Data availability

The large VCF inputs are downloaded from their original public providers rather than redistributed in Git. Canonical filenames, source URLs/download commands, experiment manifests, query files, normalized outputs, and comparison reports are maintained at <https://github.com/ecrum19/vcf-rdfizer-testing>. The archive provides run manifests and hashes for tracing the source inputs, generated artifacts, and executed queries.

4.7 Code availability

The VCF-RDFizer source is available at <https://github.com/ecrum19/VCF-RDFizer>. The target benchmark release is v3.1.0 (commit [d3b34d5](#)), distributed through PyPI, Conda-Forge, and Docker Hub; its container configuration is specified in Section 2.3. Each run records its tool commit, command, and resolved image digest. The policy demonstrator of Section 3.8, with its fixture and tests, is at <https://github.com/ecrum19/VCF-RDFizer/pull/26> and follows the v3.1.0 release. Testing scripts, the integrated per-cell run record, normalized outputs, and paper artifacts are available at <https://github.com/ecrum19/vcf-rdfizer-testing>.

4.8 Author contributions statement

E.C. and G.E. conceived the tool and the experiments. E.C. implemented the conversion workflow and executed the experiments. E.C. and R.T. analyzed the results. E.C. wrote the manuscript draft. E.C., R.T., G.E., and B.B. reviewed and edited the manuscript.

4.9 Acknowledgments

All benchmarking computation was performed using the resources of IDLab at Ghent University.

4.10 Ethics approval and consent to participate

This study used only publicly released genomic data and involved no interaction with, or recruitment of, human participants. The five Personal Genome Project files were

published by their donors under the project’s open-consent model [43]; the Genome in a Bottle and precisionFDA files are public reference materials [44, 47]; and the 1000 Genomes Phase 3 and HGSVC2 call sets are released by their consortia for research reuse [45, 46]. No attempt was made to identify any individual, and the results report only aggregate summaries and software-performance measurements. Ethics approval was therefore not sought.

4.11 Consent for publication

Not applicable. The manuscript contains no individual-level data, images or personal details; every reported figure is an aggregate count or a software-performance measurement over publicly released data.

4.12 Materials availability

No biological materials were generated or used. The digital materials needed to reproduce the study are public: VCF-RDFizer v3.1.0 (source on GitHub; packages on PyPI, Conda-Forge and Docker Hub, per-run commits and image digests in the benchmark archive); the VCF Core Vocabulary and its SHACL shapes [34]; the eleven difficult-input fixtures (`benchmarks/fixtures/awkward*.vcf`); the scripts that derive the record and sample ladders from the public corpus (`benchmarks/02_derive_ladders.sh`); and the download commands for all ten corpus files, all in <https://github.com/ecrum19/vcf-rdfizer-testing>. The corpus files themselves are not redistributed and are obtained from their original providers.

References

- [1] The Variant Call Format Specification (2024). <https://github.com/samtools/hts-specs>
- [2] Li, B., Sangkuhl, K., Keat, K., Whaley, R.M., Woon, M., Verma, S., Dudek, S., Tuteja, S., Verma, A., Whirl-Carrillo, M., Ritchie, M.D., Klein, T.E.: How to run the pharmacogenomics clinical annotation tool (pharmcat). *Clinical Pharmacology & Therapeutics* **113**(5), 1036–1047 (2023) <https://doi.org/10.1002/cpt.2790>
- [3] Souche, E., Beltran, S., Brosens, E., Belmont, J.W., Fossum, M., Riess, O., Gilissen, C., Ardeshirdavani, A., Houge, G., Van Gijn, M., Clayton-Smith, J., Synofzik, M., De Leeuw, N., Deans, Z.C., Dincer, Y., Eck, S.H., Van Der Crabben, S., Balasubramanian, M., Graessner, H., Sturm, M., Firth, H., Ferlini, A., Nabbout, R., De Baere, E., Liehr, T., Macek, M., Matthijs, G., Scheffer, H., Bauer, P., Yntema, H.G., Weiss, M.M.: Recommendations for whole genome sequencing in diagnostics for rare diseases. *European Journal of Human Genetics* **30**(9), 1017–1021 (2022) <https://doi.org/10.1038/s41431-022-01113-x>
- [4] Mitsuhashi, N., Toyo-oka, L., Katayama, T., Kawashima, M., Kawashima, S., Miyazaki, K., Takagi, T.: TogoVar: A comprehensive Japanese genetic variation

- database. *Human Genome Variation* **9**(1), 44 (2022) <https://doi.org/10.1038/s41439-022-00222-9>
- [5] Van Der Horst, E., Unni, D., Kopmels, F., Armida, J., Touré, V., Franke, W., Crameri, K., Cirillo, E., Österle, S.: Bridging Clinical and Genomic Knowledge: An Extension of the SPHN RDF Schema for Seamless Integration and FAIRification of Omics Data. (2023). <https://doi.org/10.20944/preprints202312.0373.v1>
 - [6] Danecek, P., Bonfield, J.K., Liddle, J., Marshall, J., Ohan, V., Pollard, M.O., Whitwham, A., Keane, T., McCarthy, S.A., Davies, R.M., Li, H.: Twelve years of samtools and bcftools. *GigaScience* **10**(2), 008 (2021) <https://doi.org/10.1093/gigascience/giab008>
 - [7] World Wide Web Consortium (W3C): *LinkedData*. <https://www.w3.org/wiki/LinkedData> Accessed 2026-03-04
 - [8] Iannella, R., Villata, S.: *ODRL Information Model 2.2*. W3C (2018). <https://www.w3.org/TR/odrl-model/> Accessed 2025-11-27
 - [9] Cyganiak, R., Wood, D., Lanthaler, M.: *RDF 1.1: Concepts and Abstract Syntax* (2014). <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>
 - [10] Hogan, A., Blomqvist, E., Cochez, M., D’Amato, C., Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A.: Knowledge graphs. *ACM Computing Surveys* **54**(4) (2021) <https://doi.org/10.1145/3447772>
 - [11] Belleau, F., Nolin, M.-A., Tourigny, N., Rigault, P., Morissette, J.: Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics* **41**(5), 706–716 (2008) <https://doi.org/10.1016/j.jbi.2008.03.004>
 - [12] Breit, A., Ott, S., Agibetov, A., Samwald, M.: *OpenBioLink: A benchmarking framework for large-scale biomedical link prediction* (2019) <https://doi.org/10.48550/ARXIV.1912.04616> . Publisher: arXiv Version Number: 2
 - [13] SIB Swiss Institute of Bioinformatics RDF Group Members, Altenhoff, A., Bairoch, A., Bansal, P., Baratin, D., Bastian, F., Bolleman*, J., Bridge, A., Burdet, F., Crameri, K., Dauvillier, J., Dessimoz, C., Gehant, S., Glover, N., Gnodtke, K., Hayes, C., Ibberson, M., Kriventseva, E., Kuznetsov, D., Frédérique, L., Mehl, F., Mendes De Farias*, T., Michel, P.-A., Moretti, S., Morgat, A., Österle, S., Pagni, M., Redaschi, N., Robinson-Rechavi, M., Samarasinghe, K., Sima, A.-C., Szklarczyk, D., Topalov, O., Touré, V., Unni, D., Von Mering, C., Wollbrecht, J., Zahn-Zabal*, M., Zdobnov, E.: The SIB Swiss Institute of Bioinformatics Semantic Web of data. *Nucleic Acids Research* **52**(D1), 44–51 (2024) <https://doi.org/10.1093/nar/gkad902>

- [14] Harris, S., Seaborne, A., Prud'hommeaux, E.: SPARQL 1.1 Query Language (2013). <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>
- [15] Lindenbaum, P.: JVarkit: java-based utilities for Bioinformatics (2015) <https://doi.org/10.6084/m9.figshare.1425030.v1>
- [16] Böhmer, J.K., Janssen, R.: Linked data in practice: An RDF-based approach with SPARQLing Genomics. Zenodo. Version V4 (2020). <https://doi.org/10.5281/zenodo.3903111>
- [17] Penha, E.D.S., Iriabho, E., Dussaq, A., Oliveira, D.M., Almeida, J.S.: Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics* **33**(4), 547–548 (2017) <https://doi.org/10.1093/bioinformatics/btw652>
- [18] Katayama, T., Kawashima, S., Micklem, G., Kawano, S., Kim, J.-D., Kocbek, S., Okamoto, S., Wang, Y., Wu, H., Yamaguchi, A., *et al.*: Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research* **8**, 1677 (2019) <https://doi.org/10.12688/f1000research.18238.1>
- [19] Baran, J., Durgahee, B.S.B., Eilbeck, K., Antezana, E., Hoehndorf, R., Dumontier, M.: GFVO: the Genomic Feature and Variation Ontology. *PeerJ* **3**, 933 (2015) <https://doi.org/10.7717/peerj.933>
- [20] Garrison, E., Kronenberg, Z.N., Dawson, E.T., Pedersen, B.S., Prins, P.: A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology* **18**(5), 1009123 (2022) <https://doi.org/10.1371/journal.pcbi.1009123>
- [21] Bendiken, A., Bonnal, R.J.P., Strozzi, F.: ruby-rdf/rdf-vcf: RDF VCF parser (2025). <https://github.com/ruby-rdf/rdf-vcf> Accessed 2025-03-01
- [22] Prasanna, S., Kumar, A., Rao, D., Simoes, E.J., Rao, P.: A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data* **7**, 1466391 (2025) <https://doi.org/10.3389/fdata.2024.1466391>
- [23] Larmande, P., Pittolat, B., Tando, N., Pomie, Y., Happi Happi, B.G., Guignon, V., Ruiz, M.: AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data* **26**(Suppl 1), 73 (2025) <https://doi.org/10.1186/s12863-025-01359-6>
- [24] Bodrug-Schepers, A., Chabane, H., Montoya, G., Redon, R., Gaignard, A., Serrano-Alvarado, P.: Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. *CEUR Workshop Proceedings*, Barcelona, Spain (2025). <https://hal.science/hal-04908530v2/document> Accessed 2025-11-24

- [25] Janssen, R.: SPARQLing Genomics. Open-source software. Accessed 2026-09-03 (2019). <https://gitlab.com/roelj/sparqling-genomics>
- [26] Wagner, A.H., Babb, L., Alterovitz, G., Baudis, M., Brush, M., Cameron, D.L., Cline, M., Griffith, M., Griffith, O.L., Hunt, S.E., Kreda, D., Lee, J.M., Li, S., Lopez, J., Moyer, E., Nelson, T., Patel, R.Y., Riehle, K., Robinson, P.N., Rynearson, S., Schuilenburg, H., Tsukanov, K., Walsh, B., Konopko, M., Rehm, H.L., Yates, A.D., Freimuth, R.R., Hart, R.K.: The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. *Cell Genomics* **1**(2), 100027 (2021) <https://doi.org/10.1016/j.xgen.2021.100027>
- [27] Bolleman, J.T., Mungall, C.J., Strozzi, F., Baran, J., Dumontier, M., Bonnal, R.J.P., Buels, R., Hoehndorf, R., Fujisawa, T., Katayama, T., Cock, P.J.A.: FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics* **7**(1), 39 (2016) <https://doi.org/10.1186/s13326-016-0067-z>
- [28] Eilbeck, K., Lewis, S.E., Mungall, C.J., Yandell, M., Stein, L., Durbin, R., Ashburner, M.: The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology* **6**(5), 44 (2005) <https://doi.org/10.1186/gb-2005-6-5-r44>
- [29] De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML) (2024). <https://rml.io/specs/rml/> Accessed 2025-11-24
- [30] Oo, S.M., Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In: Sattler, U., Hogan, A., Keet, M., Presutti, V., Almeida, J.P.A., Takeda, H., Monnin, P., Pirrò, G., d’Amato, C. (eds.) *The Semantic Web – ISWC 2022* vol. 13489, pp. 697–713. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_40 . Series Title: Lecture Notes in Computer Science. https://link.springer.com/10.1007/978-3-031-19433-7_40 Accessed 2025-11-24
- [31] Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A General-Purpose Data Compressor. *ACM Transactions on Information Systems* **37**(1), 1–30 (2019) <https://doi.org/10.1145/3231935>
- [32] Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web* **19**, 22–41 (2013)
- [33] Arenas-Guerrero, J., Ferrada, S.: COTTAS: Columnar triple table storage for efficient and compressed RDF management. In: *The Semantic Web – ISWC 2025. Lecture Notes in Computer Science*, vol. 16141, pp. 313–331. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-09530-5_18 . https://doi.org/10.1007/978-3-032-09530-5_18

- [34] Crum, E.: VCF Core Vocabulary. Version 2.1.3. GitHub repository. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026 (2026). <https://github.com/ecrum19/vcf-core-vocabulary/tree/8a5f23a>
- [35] Crum, E., Buelens, B., Ertaylan, G., Taelman, R.: VCF Core Vocabulary: A Semantic Foundation for RDF Representations of VCF. Manuscript in preparation for SWAT4HCLS 2027 (2026)
- [36] Menotti, L., Cazzaro, M., Rueda, M., Gut, I.G., Silvello, G.: HERO-Genomics: Bridging Genomic Data and Ontological Modelling. CEUR Workshop Proceedings, Barcelona, Spain (2025). https://www.dei.unipd.it/~silvello/papers/2025.MCS_SWAT4HCLS.pdf
- [37] Wagner, A.H., Hart, R., Babb, L., Kuzma, K., Schuilenburg, H., Stevenson, J., O'Neill, T., Epic, M.-T., Holmes, B., Moyer, E., Baudis, M., Robinson, P., Cline, M., Patel, R.Y., Freimuth, R.R.: ga4gh/vrs 2.0.1. Zenodo (2025). <https://doi.org/10.5281/zenodo.15059674> . <https://doi.org/10.5281/zenodo.15059674> Accessed 2026-03-02
- [38] RDF/HDT contributors: hdt-cpp: HDT C++ Library and Tools. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest. <https://github.com/rdfhdt/hdt-cpp>
- [39] FRINK OKN: hdtc: HDT Creator. Computer software, MIT License. <https://github.com/frink-okn/hdtc>
- [40] Diefenbach, D., Giménez-García, J.M.: HDTCat: Let's make HDT generation scale. In: The Semantic Web – ISWC 2020. Lecture Notes in Computer Science, vol. 12507, pp. 18–33. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-62466-8_2
- [41] Arenas-Guerrero, J.: pycottas: Python COTTAS library for compressing and querying RDF. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities (2026). <https://pypi.org/project/pycottas/> Accessed 2026-08-31
- [42] Crum, E.: Comunica SPARQL COTTAS: a Comunica Engine for Query Evaluation over COTTAS files. npm package @elias.crum/query-sparql-cottas. <https://github.com/ecrum19/comunica-feature-cottas>
- [43] Personal Genome Project (2005). <https://my.pgp-hms.org/public-genetic-data> Accessed 2025-11-24
- [44] NIST Genome in a Bottle (2025). <https://www.nist.gov/programs-projects/genome-bottle> Accessed 2025-11-24
- [45] The 1000 Genomes Project Consortium: A global reference for human genetic

- p variation.
- Nature*
- 526**
- (7571), 68–74 (2015)
- <https://doi.org/10.1038/nature15393>
- [46] Ebert, P., Audano, P.A., Zhu, Q., Rodriguez-Martin, B., Porubsky, D., Bonder, M.J., Sulovari, A., Ebler, J., Zhou, W., Serra Mari, R., *et al.*: Haplotype-resolved diverse human genomes and integrated analysis of structural variation. *Science* **372**(6537), 7117 (2021) <https://doi.org/10.1126/science.abf7117>
 - [47] Wagner, J., Olson, N.D., Harris, L., Khan, Z., Farek, J., Mahmoud, M., Stankovic, A., Kovacevic, V., Yoo, B., Miller, N., *et al.*: Benchmarking challenging small variants with linked and long reads. *Cell Genomics* **2**(5), 100128 (2022) <https://doi.org/10.1016/j.xgen.2022.100128>
 - [48] Danecek, P., Auton, A., Abecasis, G., Albers, C.A., Banks, E., DePristo, M.A., Handsaker, R.E., Lunter, G., Marth, G.T., Sherry, S.T., McVean, G., Durbin, R., 1000 Genomes Project Analysis Group: The variant call format and VCFtools. *Bioinformatics* **27**(15), 2156–2158 (2011) <https://doi.org/10.1093/bioinformatics/btr330>
 - [49] Pedersen, B.S., Quinlan, A.R.: cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics* **33**(12), 1867–1869 (2017) <https://doi.org/10.1093/bioinformatics/btx057>
 - [50] Taelman, R., Herwegen, J.V., Sande, M.V., Verborgh, R.: Comunica: A modular SPARQL query engine for the web. In: *Proceedings of the 17th International Semantic Web Conference*, pp. 239–255. Springer, ??? (2018). https://doi.org/10.1007/978-3-030-00668-6_15 . <https://comunica.dev/>
 - [51] Comunica Association: Querying local files from the command line (2026). https://comunica.dev/docs/query/getting_started/query_cli_file/ Accessed 2026-08-31
 - [52] Comunica Association: HDT: Querying HDT files with Comunica (2026). <https://comunica.dev/docs/query/advanced/hdt/> Accessed 2026-08-31
 - [53] Lawson, J., Cabili, M.N., Kerry, G., *et al.*: The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics* **1**(2), 100028 (2021) <https://doi.org/10.1016/j.xgen.2021.100028>
 - [54] Ashburner, M., Ball, C.A., Blake, J.A., Botstein, D., Butler, H., Cherry, J.M., Davis, A.P., Dolinski, K., Dwight, S.S., Eppig, J.T., Harris, M.A., Hill, D.P., Issel-Tarver, L., Kasarskis, A., Lewis, S., Matese, J.C., Richardson, J.E., Ringwald, M., Rubin, G.M., Sherlock, G.: Gene Ontology: tool for the unification of biology. *Nature Genetics* **25**(1), 25–29 (2000) <https://doi.org/10.1038/75556>
 - [55] Jiménez, R.C., Kuzak, M., Alhamdoosh, M., Barker, M., Batut, B., Borg, M., Capella-Gutiérrez, S., Chue Hong, N.P., Cook, M., Corpas, M., Flannery, M., García, L., Gelpí, J.L., Gladman, S., Goble, C., González Ferreiro, M., Gonzalez-Beltran, A., Griffin, P.C., Grüning, B., Hagberg, J., Holub, P., Hooft, R., Ison, J.,

Jones, S., Kaiser, F., Kim, S., Krafczyk, M., Krystalli, A., Lapp, H., Lonie, A., Marek, C., Mazanetz, M.P., Nenadic, A., Petersen, W., Plomp, E., Pomazkov, M., Ram, K., Soiland-Reyes, S., Sokolov, A., Struck, A., Suchecki, R., Vos, R., Williams, A.R., Brack, P., Muelas, M.W., Diehl, A.D., Poline, J.-B., Atwood, T.K., Mansurov, A., Soldatova, L.N., Kingsford, C., Kersey, P.J., McDowall, M.D., Østeras, M., Guijarro-Clarke, C., Crouch, S.: Four simple recommendations to encourage best practices in research software. *F1000Research* **6**, 876 (2017) <https://doi.org/10.12688/f1000research.11407.1>